

TECNOLOGÍAS DE LA INFORMACIÓN

Minería de Datos

3 créditos

Profesor Autor:



Ricardo Ordoñez Avila, Mg.

Titulaciones	Semestre
MINERÍA DE DATOS	Sexto

Tutorías: El profesor asignado se publicará en el entorno virtual de aprendizaje online.utm.edu.ec), y sus horarios de conferencias se indicarán en la sección Cronograma de Actividades

PERÍODO ABRIL 2026 - AGOSTO 2026



Índice

Tabla de contenido

Tema 1: La minería de datos descriptiva	5
1.1 Definiciones	5
1.2 Algoritmos de asociación	14
1.3 Algoritmos de agrupamiento	18
Tema 2: Técnicas de aprendizaje no supervisado: asociación y agrupamiento.	22
2.1 Algoritmo a-priori	22
2.2 Clustering jerárquico	40
2.3 Clustering particional	52
Tema 3: Métricas de calidad en modelos descriptivos.	61
3.1 Homogeneidad.	61
3.2 Separación.....	61
3.3 Jaccard.	62



✓ Organización de la lectura para el estudiante por semana del compendio

Semanas	Compendio
Semana 9	Páginas 3 – 19
Semana 10	Páginas 22 – 37
Semana 11	Páginas 40 – 57
Semana 12	Páginas 61 – 62

Resultado de aprendizaje de la Unidad III

Conocer distintas técnicas de minería de datos en las áreas de clustering, y análisis de asociación.

Unidad III

Visualización y reducción de la dimensionalidad y Clustering

La generación y el almacenamiento de grandes volúmenes de información hacen que el mismo pase desapercibido y muchas veces se pierde la oportunidad de encontrar valor en ella. La visualización de datos es el proceso de representación de datos, en formato gráfico, de una manera clara y eficaz. Se convierte en una herramienta poderosa para el análisis e interpretación de datos grandes y complejos, volviéndose un medio eficiente en la transmisión de conceptos en un formato universal (Schab et al., 2018).

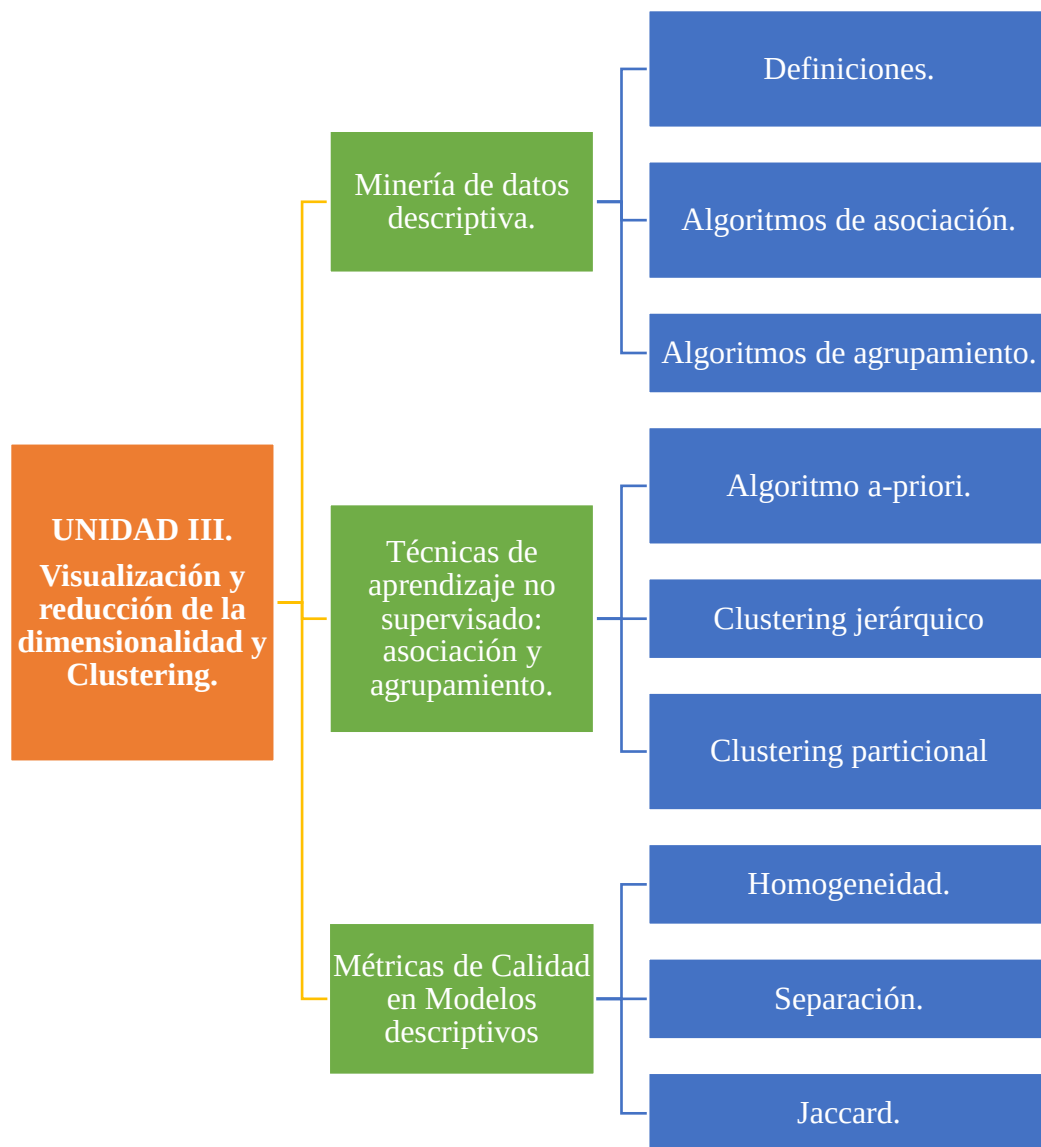
Klenzi et al. (2018) explican sobre la sobrecarga de información o exceso de datos que se sufre hoy en día es acarreada por el uso de la tecnología Web, indicando que existe una solución fácil que radica en la visualización de la información. Por medio de ésta se pueden ver patrones y conexiones, de modo que la información tiene más sentido y permite centrarse únicamente en lo que es importante. En secuencia a estas ideas, Yuk y Diamond citado en Klenzi et al. (2018), se refieren de la visualización como el estudio de cómo representar los datos usando un enfoque visual o artístico en lugar del método tradicional de reportes. Estos autores consideran que ser útil, usable y deseable son características recomendables de toda visualización, y a ellas les agregan: ser visualmente atractivas, escalables, proveer al usuario de información correcta y ser accesibles en cualquier momento. Como todo proceso, la minería de datos también se lleva a cabo en distintas fases, y como sucede en la minería tradicional, también existen varias técnicas de extracción (en nuestro caso, para extraer esa información considerada relevante para el negocio y mostrarla de un modo comprensible).

Los pasos que incluye cualquier proceso de data mining empiezan con una selección de los datos a tratar en función de variables de predicción y cálculo, es decir, de un conjunto de variables objetivas y otro de variables independientes: las primeras son las que guían la elección en función de los objetivos que se persiguen con el análisis, y las segundas determinan de qué modo se llevará a cabo el proceso. Tras ello, se analizan las propiedades del conjunto de datos seleccionados para detectar patrones, tendencias, valores atípicos y datos descartables con nulo valor informacional. Este análisis previo guiará el posterior procesamiento de los datos, que permitirá clasificarlos y segmentarlos en función del modelo predictivo elegido, y tras lo cual se elaborarán modelos de conocimiento gracias a la identificación de patrones de comportamiento, y de elementos de asociación y disociación entre las distintas variables usadas en el análisis previo.

Finalmente, una vez obtenido el o los modelos de conocimiento mediante el uso de las distintas técnicas aplicables, se procede a la validación de los mismos tras compararlos e interpretarlos, y a la elección del más satisfactorio según los resultados obtenidos. Si ningún modelo satisface las expectativas de conocimiento esperadas, el proceso se repite de nuevo cambiando variables y adoptando técnicas distintas a las usadas en los procesos anteriores, hasta obtener un modelo la explotación del cual satisfaga las necesidades de conocimiento corporativo expresadas (Logicalis, 2014).

La visualización de datos y, en concreto, el uso de las herramientas que cumplen este cometido es fundamental para agilizar el proceso y ahorrar tiempo y esfuerzos a los expertos que deben determinar, con la máxima rapidez y eficiencia, si los modelos obtenidos se ajustan con lo esperado. En esa comparativa entre modelos y su evaluación para determinar si son lo suficientemente satisfactorios es donde entran en juego las herramientas de visualización de datos, que simplifican y agilizan la tarea de los expertos permitiendo optimizar el proceso de data mining, reduciendo el tiempo empleado para llevarlo a cabo y minimizando los riesgos asociados a una mala interpretación de los resultados obtenidos. Por tanto, la minería de datos y, por consiguiente, la visualización de datos, están íntimamente relacionados con una correcta gestión de los datos y la información.

Ejes Temáticos



Tema 1: La minería de datos descriptiva

1.1 Definiciones

Las técnicas descriptivas de la minería de datos están orientadas a describir los datos o el fenómeno que subyace sobre los mismos, y en líneas generales se aplica del lado del aprendizaje no supervisado. En este último paradigma los datos y propiedades de los conjuntos a analizar se caracterizan por la ausencia de una etiqueta, clase o salida dentro de los mismos. Indicar también que existe un conjunto

de técnicas descriptivas que son capaces de analizar y describir las relaciones de los datos bajo el aprendizaje supervisado.

A día de hoy, existe un gran reto dentro del análisis y descripción de datos dentro de la Ciencia de Datos ya que estamos ante la necesidad de analizar grandes volúmenes de datos, que en algunas situaciones son volátiles, cambiantes, heterogéneos y que pueden proceder de distintas fuentes (DaSCI, 2022).

En el proceso de minería de datos, se debe entender el problema que se desea resolver, de manera que se pueda determinar el tipo de modelo a aplicar, sea este, un modelo predictivo, o un modelo descriptivo. Para cualquiera de los casos, finalmente, se elige el algoritmo de minería que resuelva la tarea y/u obtenga el modelo (Marin, 2022).

Una de estas tareas, al referirnos a las de tipo descriptivas, es la de agrupamiento. En esta, a diferencia de la clasificación, se busca agrupar a los individuos maximizando el grado de similitud entre las instancias de un mismo grupo y minimizando la similitud entre los distintos grupos. Los grupos pueden ser o no disjuntos. Por ejemplo, una librería que ofrece sus servicios a través de la red usa el agrupamiento para identificar grupos de clientes en base a sus preferencias de compras que le permita dar un servicio más personalizado. Si un cliente se interesa por un libro, el sistema identifica a que grupo pertenece y le recomienda otros libros. Mientras que, las reglas de asociación, tienen por objetivo determinar relaciones no evidentes entre atributos categóricos. No necesariamente las reglas son del tipo causa-efecto. Por ejemplo: El clásico caso de la cesta de compra para organizar los productos físicamente en el supermercado (Marin, 2022).

1.1.1 Reducir la dimensionalidad

Los métodos de reducción de dimensionalidad son algoritmos que mapean el conjunto de los datos a subespacios derivados del espacio original, de menor dimensión, que permiten hacer una descripción de los datos a un menor costo. Por su importancia, son ampliamente usados en procesos asociados a aprendizaje de máquina.

Una de las técnicas de preprocesado para modelos de aprendizaje supervisado es la reducción de la dimensionalidad, que no es más que la reducción del número de variables en una colección de datos (Lafuente, 2018).

Los métodos de reducción de dimensionalidad son algoritmos que mapean el conjunto de los datos a subespacios derivados del espacio original, de menor dimensión, que permiten hacer una descripción de los datos a un menor costo. Por su importancia, son ampliamente usados en procesos asociados a aprendizaje de máquina (Arroyo-Hernández, 2016). A mayor dimensionalidad del problema, minería de datos ofrece mejores soluciones. Cuantas más variables entren en el problema, más difícil resulta encontrar hipótesis de partida interesantes (Beltrán, 2022).

La dimensión de los datos es el número de variables, también conocido como atributos o características, que son medidas en cada observación. Por tanto, la reducción de la dimensionalidad, sobre un conjunto de datos con un número elevado de variables, hace referencia al proceso de reducción del número de dimensiones de los datos originales generando un conjunto de datos con una capacidad predictiva similar, pero con menor número de variables. El objetivo general de la reducción de la dimensionalidad de un conjunto de datos es eliminar variables que sean irrelevantes o redundantes. Sin embargo, realizando esto podemos perder gran cantidad de información valiosa para resolver el problema. Por este motivo, la clave consiste en tratar de reducir la dimensionalidad sin sacrificar la información más importante del conjunto de datos (Soriano, 2016). Como algunas de las características especiales de los datos, aparte del gran volumen, ¿por qué las técnicas de aprendizaje automático y estadísticas no son directamente aplicables? Algunas de estas razones se justifican por:

- Los datos residen en el disco.
- No se pueden escanear múltiples veces.
- Algunas técnicas de muestreo no son compatibles con algoritmos no incrementales.
- ***Muy alta dimensionalidad (muchos campos).***
- Evidencia Positiva.
- Datos Imperfectos.

Aunque algunos se aplican casi directamente, el interés en la investigación en minería de datos está en su adaptación (Beltrán, 2022).

Reducción de dimensionalidad y linealidad: los métodos de reducción de la dimensionalidad son el proceso que transforma un conjunto de datos con dimensionalidad m a un nuevo conjunto Q con dimensionalidad m' ($m' < m$), de forma tal que conserve la mayor información intrínseca posible; además, su principal distinción es que pueden ser desarrollados desde el supuesto o no de la linealidad.

El análisis de componentes principales (ACP) es una técnica lineal que se utiliza para la eliminación de la redundancia de los datos (Shlens, 2005, como se citó en Arroyo-Hernández, 2016). Es ampliamente usado, sin embargo, su mayor limitación se basa en el supuesto de linealidad. El ACP es uno de los métodos de extracción de características más empleado, estos realizan una transformación completa de los datos originales de alta dimensión a un conjunto de menor dimensión tratando de retener la mayor cantidad de información posible. Existen multitud de algoritmos de extracción de características y se pueden clasificar en técnicas supervisadas y no supervisadas. Las técnicas supervisadas son aquellas que realizan la transformación de los datos originales teniendo en cuenta la solución del problema. Por el contrario, las técnicas no supervisadas realizan la transformación solo teniendo en cuenta los datos (Soriano, 2016).

El ACP permite un cambio de base a una de menor dimensionalidad sobre X a través de la ecuación de transformación $Y = PX$, donde P es una matriz ortogonal denominada matriz de representación. El objetivo es determinar la matriz P que permita que la nube de datos pueda ser proyectada a un espacio de menor dimensión.

La estrategia es buscar P de forma que se garantice la no correlación entre vectores de Y , es decir, $C_{ij} \in C_Y$, $i \neq j$ sean nulos. Si la correlación entre las distintas muestras es nula, se elimina la redundancia y el subespacio de datos puede ser descrito por P . De lo contrario, cada entrada C_{ij} que corresponda a valores grandes que representará alta redundancia de las observaciones i y j , y por ende, habrá el ruido presente. Según el mismo autor, el algoritmo para hallar P inicia con el centrado y estandarizado de los datos.

$$X, C_x = \frac{1}{n} XX^T$$

Luego, se calcula la matriz de covarianza de X , $C_x = \frac{1}{n} XX^T$ que es simétrica y diagonalizable, y que cuantifica la covarianza entre las mediciones. Luego, se obtiene los vectores propios de C_x , que son elegidos como columnas vectores de P , ordenados de acuerdo con el valor propio y que sirven de nuevas coordenadas del sistema donde es maximizada la varianza. Se elige el número adecuado de vectores propios que son denominados componentes principales y que describen la información del conjunto de datos de acuerdo con su coeficiente de inercia, el cual indica el porcentaje de esta, presente en cada componente principal (Arroyo-Hernández, 2016). Desde el punto de vista geométrico, el ACP puede ser visto como una rotación de los ejes originales sobre sus medias, en la figura 1, se presenta la rotación de los datos.

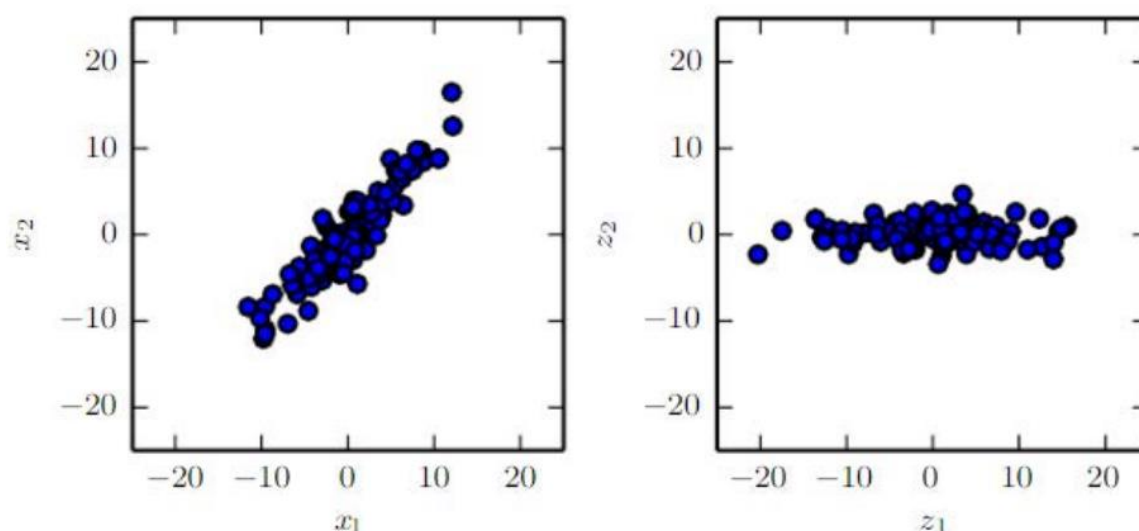


Figura 1. Rotación de los datos.

El primer componente principal es definido por el método donde el conjunto de datos presenta una mayor varianza, el segundo componente principal es definido donde los datos presentan la segunda mayor varianza, y así sucesivamente. Hay que tener en cuenta que el número máximo de componentes principales que se obtienen mediante ACP siempre será igual al número de variables del conjunto de datos, aunque como el objetivo es reducir la dimensionalidad habrá que reducir el número de componentes seleccionando aquellos que expliquen la mayor cantidad de variabilidad de los datos.

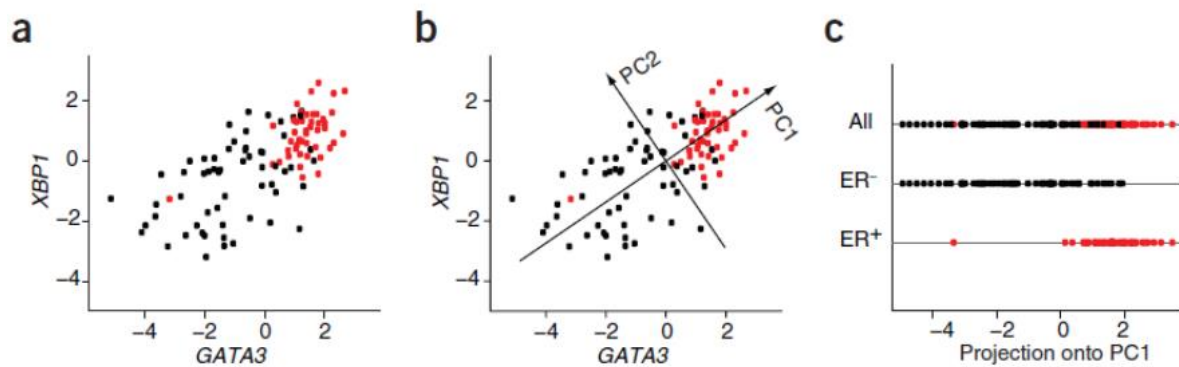


Figura 2. Ejemplo de funcionamiento de ACP.

En este ejemplo de la figura 2, los datos están etiquetados con dos clases distintas (ER+ y ER-), podemos visualizarlo en la figura (a) como puntos rojos (ER+) y negros (ER-). En la figura (b) vemos los componentes principales (PC1 y PC2) sobre el conjunto de datos. En la figura (c) se muestran los datos sobre una única dimensión, es decir, se ha eliminado la segunda componente (PC2); se puede apreciar que se ha perdido información quitando un componente, pero es posible extraer conclusiones visualizando como los datos de las dos clases están bastante separados, es decir, en extremos opuestos de la gráfica. Nótese que si los datos son proyectados sobre el PC2 las conclusiones que se podrían obtener serían engañosas, ya que veríamos los datos de las clases juntos (Soriano, 2016).

Esta técnica presenta como principal ventaja que retiene las características del conjunto de datos que contribuyen más a su varianza. Sin embargo, tiene el inconveniente que no ofrece resultados satisfactorios en muchos problemas supervisados de clasificación o regresión al tratarse de una técnica no supervisada.

Softtek (2021), explica algunos de los principales beneficios que genera aplicar la técnica de reducción de dimensionalidad son los siguientes:

- Reducir las dimensiones de las características implica una reducción del espacio requerido para almacenar el conjunto de datos, porque este también se reduce.
- El tiempo de entrenamiento de modelos es menor para dimensiones reducidas.
- Se facilita la visualización de datos más rápidamente gracias a la reducción de características del conjunto de datos.

- Desaparecen las características redundantes en el ámbito de la multicolinealidad.
- La reducción de dimensionalidad también presenta ciertos inconvenientes que se mencionan a continuación, aunque las ventajas son mayores:
- Se pueden perder algunos datos debido a la reducción de la dimensionalidad.
- En la técnica de reducción de dimensionalidad de ACP, en ocasiones no se conocen los componentes principales que se deben considerar.

Ejemplo de análisis de componentes principales en R.

La función **prcomp** puede usar la matriz de correlación (con datos estandarizados) o la matriz de covarianzas. También es posible hacer ACP con la librería **caret**.

```
ir.pca <- prcomp(iris[-5], scale=T)
# prcomp(iris.standardized, scale=T)
# prcomp(iris.standardized)
# princomp(iris.standardized, cor=T)
print(ir.pca)
## Standard deviations:
## [1] 1.7083611 0.9560494 0.3830886 0.1439265
##
## Rotation:
##           PC1          PC2          PC3          PC4
## Sepal.Length 0.5210659 -0.37741762 0.7195664 0.2612863
## Sepal.Width  -0.2693474 -0.92329566 -0.2443818 -0.1235096
## Petal.Length 0.5804131 -0.02449161 -0.1421264 -0.8014492
## Petal.Width 0.5648565 -0.06694199 -0.6342727 0.5235971
summary(ir.pca)
## Importance of components:
##           PC1          PC2          PC3          PC4
## Standard deviation 1.7084 0.9560 0.38309 0.14393
## Proportion of Variance 0.7296 0.2285 0.03669 0.00518
## Cumulative Proportion 0.7296 0.9581 0.99482 1.00000
```

Esta es la manera de obtener sólo las desviaciones estándar de los componentes

```
ir.pca$sdev
## [1] 1.7083611 0.9560494 0.3830886 0.1439265
```

Con `ir.pca$rotation` se obtienen los pesos de los componentes

	PC1	PC2	PC3	PC4
Sepal.Length	0.5210659	-0.3774176	0.7195664	0.2612863
Sepal.Width	-0.2693474	-0.9232957	-0.2443818	-0.1235096
Petal.Length	0.5804131	-0.0244916	-0.1421264	-0.8014492
Petal.Width	0.5648565	-0.0669420	-0.6342727	0.5235971

En este momento ya sabemos que componentes capturan la mayoría de la varianza. Vamos a hacerlo más visual.

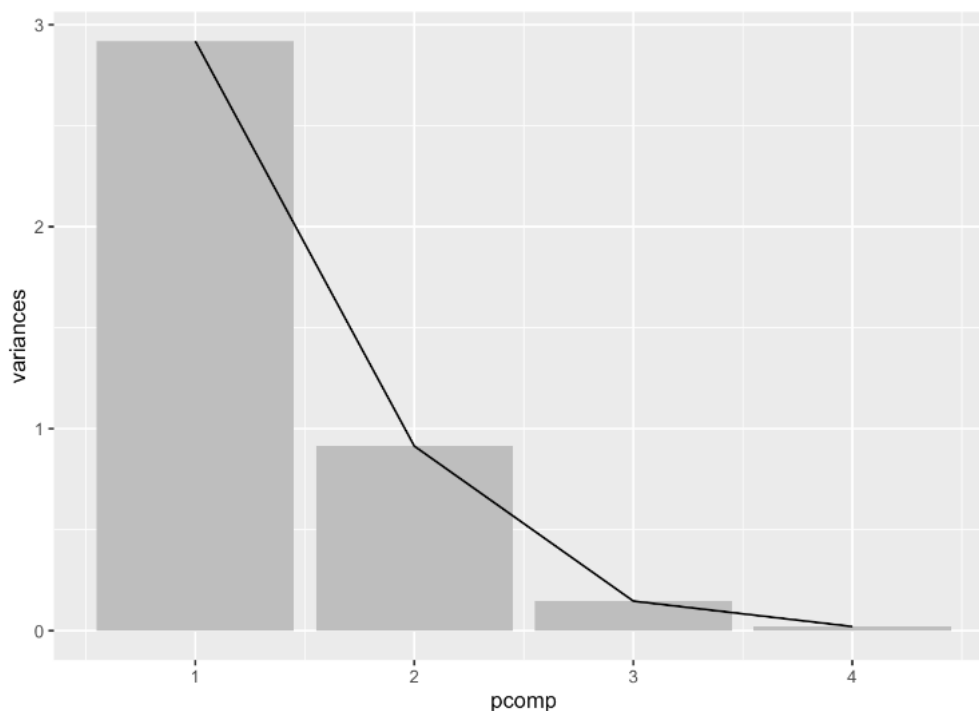


Figura 3. Gráfica de varianzas de los componentes.

Podemos observar que los componentes 1 y 2 son los más significativos. Por un resultado anterior sabemos que capturan casi el 96% de las varianzas de las variables.

La siguiente figura nos ayuda a visualizar el método de Componentes Principales

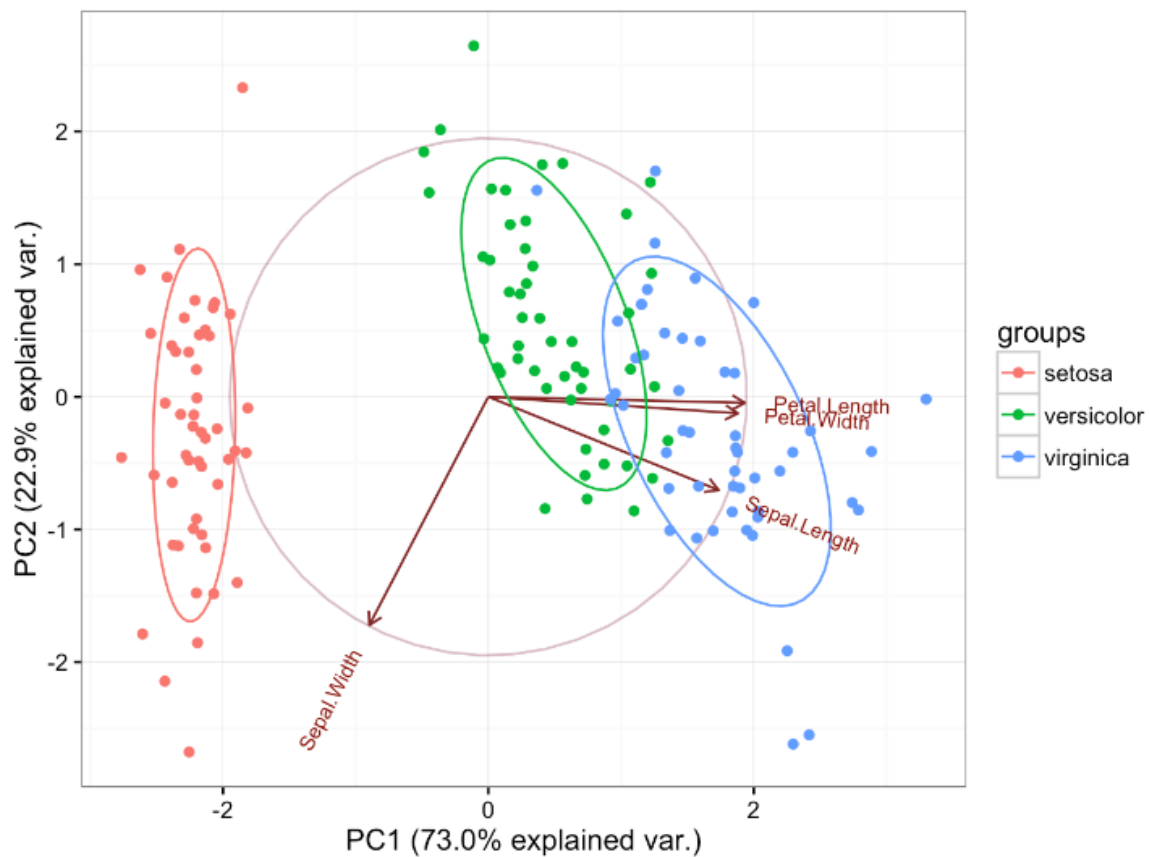


Figura 4. Visualización de los componentes y las observaciones.

Podemos notar que la anchura y la longitud de los pétalos están altamente correlacionadas y su variabilidad a través de las tres especies Iris se explica principalmente por el componente 1, que también explica una gran parte de la variabilidad en la longitud del sépalo.

La siguiente figura es una simplificación de la Figura 4 y ayuda a la interpretación de los componentes 1 y 2 en relación a las variables numéricas.

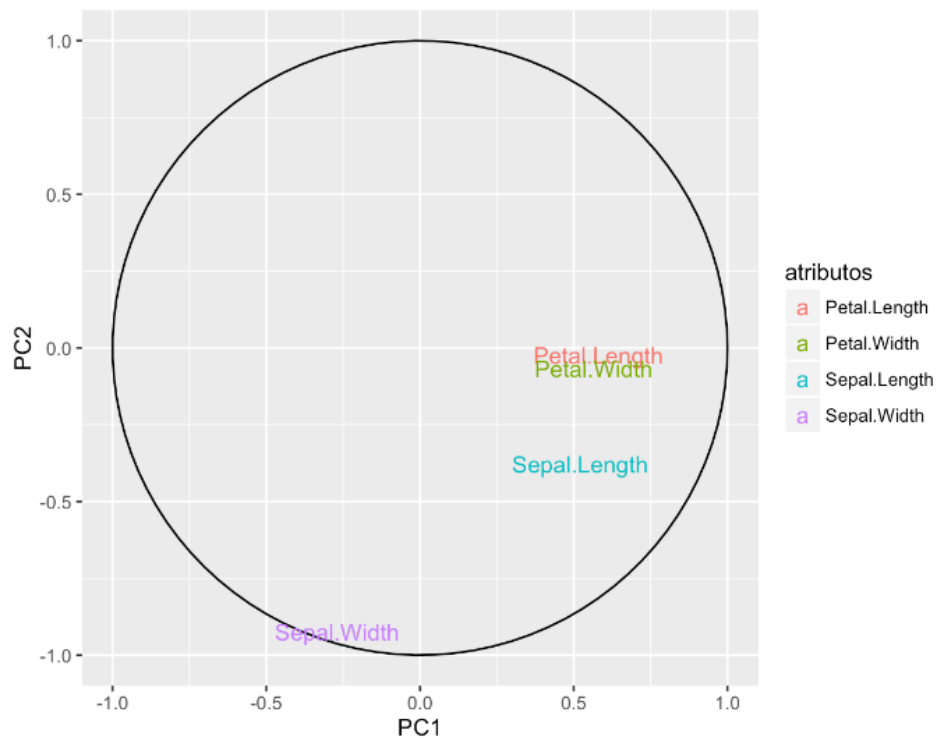


Figura 5. Interpretación de los componentes principales

Para este caso los componentes 1 y 2 resumen el modelo de la base Iris. Hay que tener presente que PC1 y PC2 son variables sintéticas y no correlacionadas entre sí (Solis, 2016).

1.2 Algoritmos de asociación

La minería de las reglas de asociación es un área muy importante dentro de la minería de datos. Es un proceso no supervisado que tiene la finalidad de encontrar las reglas de asociación que se encuentran en las instancias de base de datos, su principal desarrollo ha sido en el área de los negocios, y en especial en los negocios minoristas, la base de datos que analiza la minería de las reglas de asociación son las transacciones que quedan registradas en una base de datos en las tiendas minoristas, en cada una de las transacciones el cliente lleva cierta cantidad de artículos de la canasta de artículos de venta que tiene ese negocio minorista, con esta información capturada, los algoritmos de minería de las reglas de asociación, lleva a cabo un análisis en donde determina cuales artículos se venden unos con otros, de esta asociación de artículos (ítem) se obtienen las reglas de asociación, debido a la gran cantidad de artículos que están tiendas tienen, existe una explosión de las reglas

que se pueden generar, para obtener solamente las reglas más importantes se usan dos parámetros, el soporte mínimo y la confianza mínima que sirven para seleccionar cuales son las reglas de asociación más interesantes (Sáenz et al., 2017).

Una regla de asociación es una implicación de la forma $A \rightarrow B$, esta regla de asociación dice que cuando se compra el ítem. es probable que se compre el ítem, tanto como pueden estar formados por uno o varios ítems. Otra interpretación que se le puede dar a la regla $A \rightarrow B$ es que cuando se cumple la condición. se lleva a cabo la acción (Sáenz et al., 2017).

Este tipo de técnicas se emplea para establecer las posibles relaciones o correlaciones entre distintas acciones o sucesos aparentemente independientes; pudiendo reconocer como la ocurrencia de un suceso o acción puede inducir o generar la aparición de otros. Son utilizadas cuando el objetivo es realizar análisis exploratorios, buscando relaciones dentro del conjunto de datos. Las asociaciones identificadas pueden usarse para predecir comportamientos, y permiten descubrir correlaciones y co-ocurrencias de eventos.

Debido a sus características, estas técnicas tienen una gran aplicación práctica en muchos campos como, por ejemplo, el comercial ya que son especialmente interesantes a la hora de comprender los hábitos de compra de los clientes y constituyen un pilar básico en la concepción de las ofertas y ventas cruzada, así como del "merchandising". En otros entornos como el sanitario, estas herramientas se emplean para identificar factores de riesgo en la aparición o complicación de enfermedades. Para su utilización es necesario disponer de información de cada uno de los sucesos llevados a cabo por un mismo individuo o cliente en un determinado período temporal.

Por lo general esta forma de extracción de conocimiento se fundamenta en técnicas estadísticas, como los análisis de correlación y de variación (Molina & García, 2006). Los algoritmos de reglas de asociación tienen como objetivo encontrar relaciones dentro un conjunto de transacciones, en concreto, *items* o atributos que tienden a ocurrir de forma conjunta. En este contexto, el término transacción hace

referencia a cada grupo de eventos que están asociados de alguna forma, por ejemplo:

- La cesta de la compra en un supermercado.
- Los libros que compra un cliente en una librería.
- Las páginas web visitadas por un usuario.
- Las características que aparecen de forma conjunta.

A cada uno de los eventos o elementos que forman parte de una transacción se le conoce como *item* y a un conjunto de ellos *itemset*. Una transacción puede estar formada por uno o varios *items*, en el caso de ser varios, cada posible subconjunto de ellos es un *itemset* distinto. Por ejemplo, la transacción $T = \{A, B, C\}$ está formada por 3 *items* (A , B y C) y sus posibles *itemsets* son: $\{A, B, C\}$, $\{A, B\}$, $\{B, C\}$, $\{A, C\}$, $\{A\}$, $\{B\}$ y $\{C\}$.

Una regla de asociación se define como una implicación del tipo “si X entonces Y ” ($X \Rightarrow Y$), donde X e Y son *itemsets* o *items* individuales. El lado izquierdo de la regla recibe el nombre de antecedente o *left-hand-side (LHS)* y el lado derecho el nombre de consecuente o *right-hand-side (RHS)*. Por ejemplo, la regla $\{A, B\} \Rightarrow \{C\}$ significa que, cuando ocurren A y B , también ocurre C .

1.2.1 Algoritmo a priori

Uno de los algoritmos más utilizado es el algoritmo A priori. El proceso del algoritmo Apriori empieza con la obtención de los llamados conjuntos de ítems frecuentes, los cuales son aquellos conjuntos formados por los ítems cuyo soporte obtenido de la base de datos es superior al soporte mínimo solicitado por el usuario. Debido al amplio uso del algoritmo Apriori, desde que se formalizó la inducción de reglas de asociación, la obtención de los conjuntos de ítems frecuentes es una tarea común en dichos algoritmos.

Agrawal 1994 citado en Sáenz et al., (2017), en su algoritmo Apriori menciona que todo subconjunto de un conjunto de ítems frecuentes también será un conjunto de ítems frecuentes. Por lo tanto, el algoritmo Apriori obtiene en primer lugar los conjuntos de ítems frecuentes de tamaño 1 y, luego, los de tamaño 2 y así sucesivamente hasta que no se encuentren más conjuntos cuyos ítems no tengan el

soporte mayor al soporte mínimo. Un ejemplo de cómo funciona el algoritmo Apriori es el siguiente, supongamos que tenemos una conjunto de transacciones en donde cada transacciones puede estar formada por uno o varios de los siguientes ítems; {a}, {b},{c},{d},{e}. Los conjuntos de ítems que el algoritmo buscara se muestran en la figura 6.

Conjunto de ítems	Núm. De transacciones	
Conjunto de un solo ítem	{a}, {b}, {c}, {d}, {e}	
Conjuntos de dos ítems	{a, b}, {a, c}, {a, d}, {a, e} {b, c}, {b, d}, {b, e} {c, d}, {c, e} {d, e}	
Conjuntos de tres ítems	{a, b, c}, {a, b, d}, {a, b, e} {a, c, d}, {a, c, e} {a, d, e} {b, c, d}, {b, c, e} {b, d, e} {c, d, e}	
Conjuntos de cuatro ítems	{a, b, c, d}, {a, b, c, e} {a, b, d, e} {a, c, d, e} {b, c, d, e}	
Conjunto con 5 ítems	{a, b, c, d, e}	

Figura 6. Conjuntos de ítems.

Los conjuntos de un solo ítem de la figura 6 son los obtenidos de una pasada en la base de datos y son los ítems cuyo soporte calculado en esa pasada es superior al soporte mínimo propuesto por el usuario.

Con esos conjuntos de un solo ítem se generan los conjuntos de dos ítems, para esto se combina el ítem {a} con el ítem {b} para formar el conjunto de dos ítems {a,b}, con este conjunto se calcula su soporte y si es mayor que el soporte mínimo antes definido, entonces este ítem de 2-item forma parte de los conjuntos de conjuntos de 2-item, ese proceso se realiza también para los ítem {a} y {b}, y así sucesivamente.

Este proceso continua ahora con los conjuntos de tres ítems que se obtienen combinando los conjuntos de 1 ítem con los conjuntos de 2 ítems, un ejemplo seria combina el ítem {a} del conjunto de 1 ítem con un conjunto de 2 ítems, por ejemplo {b,c} para formar el conjunto {a,b,c}, este conjunto se buscado en la base de datos de transacciones, y si su soporte es mayor al soporte mínimo entonces forma parte del

conjunto de tres ítems. Considerando solamente el conjunto de tres ítems, las reglas de que podrían generar serían las siguientes:

$$\begin{aligned}\{a, b\} &\rightarrow \{c\} \\ \{a, c\} &\rightarrow \{b\} \\ \{b, c\} &\rightarrow \{a\}\end{aligned}$$

Por lo que este algoritmo generaría una gran cantidad de reglas de asociación. De acuerdo con Neves 2008 citado en Sáenz et al., (2017), se recomienda que, como parámetros de entrada del algoritmo, se defina un valor bajo para el soporte y un valor elevado para la confianza.

De esta forma, en primer lugar, se genera una gran cantidad de reglas y, posteriormente, se verifica la cohesión de las mismas a través de la medida de confianza. Una regla de asociación con un valor de confianza bajo no expresará un patrón de comportamiento en los datos y, por otra parte, un valor de soporte muy elevado probablemente llevaría a la pérdida de patrones.



Sabías que: El algoritmo Apriori es un algoritmo eficiente que escanea la base de datos solo una vez. Reduce considerablemente el tamaño de los conjuntos de elementos de la base de datos proporcionando un buen rendimiento. Por lo tanto, la minería de datos ayuda a los consumidores y las industrias a mejorar el proceso de toma de decisiones.

1.3 Algoritmos de agrupamiento

Clustering (segmentación), también llamada agrupamiento, permite la identificación de tipologías o grupos donde los elementos guardan gran similitud entre sí y muchas diferencias con los de otros grupos. Así se puede segmentar el colectivo de clientes, el conjunto de valores e índices financieros, el espectro de observaciones astronómicas, el conjunto de zonas forestales, el conjunto de empleados y de sucursales u oficinas, etc. La segmentación está teniendo mucho interés desde hace ya tiempo dadas las importantes ventajas que aporta al permitir el tratamiento de grandes colectivos de forma pseudo-particularizada, en el más idóneo punto de

equilibrio entre el tratamiento individualizado y aquel totalmente masificado. Las herramientas de segmentación se basan en técnicas de carácter estadístico, de empleo de algoritmos matemáticos, de generación de reglas y de redes neuronales para el tratamiento de registros.

Para otro tipo de elementos a agrupar o segmentar, como texto y documentos, se usan técnicas de reconocimiento de conceptos. Esta técnica suele servir de punto de partida para después hacer un análisis de clasificación sobre los clusters. La principal característica de esta técnica es la utilización de una medida de similitud que, en general, está basada en los atributos que describen a los objetos, y se define usualmente por proximidad en un espacio multidimensional. Para datos numéricos, suele ser preciso preparar los datos antes de realizar data mining sobre ellos, de manera que en primer lugar se someten a un proceso de estandarización. Una de las técnicas empleadas para conseguir la normalización de los datos es utilizar la medida z (z -score) que elimina las unidades de los datos (Molina & García, 2006).

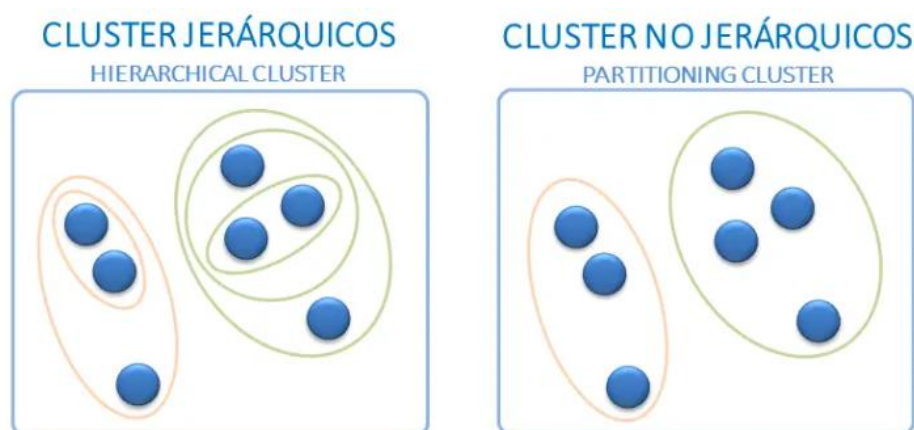


Figura 7. Análisis Clúster jerárquico vs Análisis Clúster no jerárquico (Calvo, 2018).

El Análisis clúster es una técnica de análisis exploratorio utilizada para resolver problemas de clasificación. Se encuadra en los métodos de aprendizaje no supervisado. Su objetivo es ordenar objetos (definidos por un conjunto de variables) en grupos de forma que los miembros del grupo sean lo más homogéneos posibles y los más heterogéneos entre miembros de distintos grupos. El análisis de clúster permite descubrir asociaciones y estructuras que no son evidentes a priori pero que pueden ser útiles una vez que se han detectado. Las estructuras encontradas pueden utilizarse para la definición formal de un esquema de clasificación (taxonomía). Los

métodos más utilizados de clusterización se dividen en dos grandes grupos: jerárquicos y no jerárquicos (Calvo, 2018).



Figura 7. Clasificación de métodos jerárquicos y no jerárquicos (Calvo, 2018)..

Los **métodos jerárquicos** o agrupamientos jerárquicos van generando grupos en cada una de las fases del proceso buscando el número de clúster que hacer una agrupación óptima. El agrupamiento jerárquico es capaz de fijar por si solos el número de clústers, por ello se pueden utilizar de forma exploratoria y posteriormente aplicar un análisis no jerárquico con el número de clúster ya fijado.

Las estrategias para conseguir este objetivo se dividen en: estrategias aglomerativas y divisivas. A estos métodos se les puede aplicar tanto las estrategias aglomerativas como divisivas:

- Método del vecino más próximo (nearest neighbor clustering) o Enlace simple (Single linkage clustering)
- Método del vecino más lejano (furthest neighbor clustering) o Enlace completo (Complete Linkage clustering)

- Método de agrupación de vinculación promedio (Average linkage clustering) o Vinculación inter-grupo (unweighted Pair-group arithmetic averages (UPGMA))
- Método de Ward o Varianza mínima (Minimum variance clustering)
- Método del Centroide
- Método de la Mediana

Los **métodos no jerárquicos** categorizan los elementos según un número de clúster dado. Necesitan que el número de particiones esté fijado a priori.

- Métodos de re-asignación
 - k-means (centroides)
 - k-medoids o PAM (medioides)
 - Clara (medioides)
 - Quick-cluster (centroides)
 - Método de Forgy (centroides)
 - Nubes dinámicas
- Métodos de búsqueda de densidad
 - Análisis Modal (aproximación tipológica)
 - Método Taxmap (aproximación tipológica)
 - Método de Fortin (aproximación tipológica)
 - Método de Wolf (aproximación probabilística)
- Métodos directos
 - Block-Clustering
- Métodos reducidos
 - Análisis Factorial tipo Q.

Tema 2: Técnicas de aprendizaje no supervisado: asociación y agrupamiento.

2.1 Algoritmo a-priori

A pesar de ser muy utilizado actualmente, la ejecución del algoritmo Apriori es muy costosa, pues como se pudo observar anteriormente, el algoritmo genera muchas combinaciones de conjuntos de ítems y realiza posteriormente repetidas búsquedas por conjuntos de ítems frecuentes.

Parámetros de las reglas de asociación.

Los parámetros de las reglas de asociación sirven para medir que tan válidas y representativa son las reglas de asociación con respecto al conjunto de datos que se está analizando. El algoritmo Apriori genera una gran cantidad de reglas con muy pocos ítems en la base de datos, de las cuales debemos de seleccionar cuales son las reglas más válidas y representativas a tomar en cuenta en base a los parámetros de las mismas. Estos parámetros nos ayudaran a seleccionar de la gran cantidad de reglas de asociación generadas por el algoritmo, dependiendo de la importancia de las mismas en base a los valores de estos parámetros (Sáenz et al., 2017). Los principales parámetros de calidad de las reglas de asociación son el Soporte y la Confianza, la descripción de cada uno de ellos es la siguiente.

Soporte

El soporte de un ítem es la frecuencia con la cual este ítem se encuentra en las transacciones dividido entre el número de transacciones.

$$\text{Soporte}(A) = \frac{\text{Numero de transacciones que contienen el ítem } A}{\text{Numero de transacciones de la base de datos}}$$

Para obtener el soporte de una regla de decisión, por ejemplos $A \rightarrow B$, se obtiene con la siguiente ecuación

$$\text{Soporte}(A \rightarrow B) = \frac{\text{Numero de transacciones que contienen los ítems } A \text{ y } B}{\text{Numero de transacciones de la base de datos}}$$

Confianza

La medida de confianza de una regla de decisión ($A \rightarrow B$) es la división entre el soporte de la regla de decisión entre el soporte del antecedente de la regla de decisión, esto está representado por la siguiente ecuación:

$$conf(A \rightarrow B) = \frac{soporte(A, B)}{soporte(B)}$$

El paquete Arules

El paquete de R **arules** implementa el algoritmo *Apriori* para la identificación de *itemsets* frecuentes y la creación de reglas de asociación a través de la función **apriori()**. También implementa el algoritmo *Eclat* con la función `eclat()` (Joaquín, 2018).

Tanto `apriori()` como `eclat()` reciben como argumento un objeto *transaction* con los datos de las transacciones, un argumento *parameter* que determina las características de los *itemsets* o reglas generadas (por ejemplo, el soporte mínimo) y un argumento *control* que determina el comportamiento del algoritmo (por ejemplo, ordenación de los resultados).

La función `apriori()` también se incluye el argumento *aparence* que impone restricciones sobre las reglas generadas, por ejemplo, crear solo reglas que contengan un determinado *item*. El resultado de ambas funciones es un objeto de tipo *association* que puede ser manipulado con toda una serie de funciones que ofrece **arules**. Entre las principales destacan:

- `summary()`: muestra un resumen de los resultados.
- `inspect()`: muestra los resultados.
- `length()`: número de elementos (reglas o *itemsets*) almacenados.
- `items()`: extrae los *items* que forman un *itemset* o una regla. En el caso de reglas, combina los *items* de antecedente (LHS) y del consecuente (RHS).
- `sort()`: ordena los resultados.
- `subset`: filtrado de los resultados.

Ejemplo cesta de la compra

En Joaquín (2018), se plantea el ejemplo de la cesta de compra, en el que se dispone del registro de todas las compras que se han realizado en un supermercado. El objetivo del análisis es identificar productos que tiendan a comprarse de forma conjunta para así poder situarlos en posiciones cercanas dentro de la tienda y maximizar la probabilidad de que los clientes compren. Para este ejemplo se emplea el set de datos Groceries del paquete arules, que contiene un registro de todas las ventas realizadas por un supermercado durante 30 días. En total se dispone de 9835 transacciones formadas por combinaciones de 169 productos.

El objeto Groceries almacena la información en un formato propio de este paquete (descrito más adelante). Para representar mejor lo que suele ocurrir en la práctica, se ha reestructurado la información en formato de tabla. Los datos pueden encontrarse en Github.

```
library(tidyverse)
datos <- read_csv(file = "./datos/datos_groceries.csv", col_names = TRUE)
head(datos)
```

id_compra	item
<dbl>	<chr>
1	citrus fruit
1	semi-finished bread
1	margarine
1	ready soups
2	tropical fruit
2	yogurt

6 rows

Cada línea del archivo contiene la información de un *item* y el identificador de la transacción (compra) a la que pertenece. Esta es la estructura en la que comúnmente se almacenan los datos dentro de una base de datos y que, en el ámbito de las transacciones, recibe el nombre de tabla larga o *single*.

Otra forma en la que se pueden encontrar los datos de transacciones es en formato matriz o tabla ancha, en el que cada fila contiene todos los *items* que forman parte de una misma transacción, este formato recibe el nombre de cesta o *basket*. Tal y como se ha definido previamente, el concepto de transacción hace referencia al conjunto de *items* o eventos que ocurren de forma conjunta.

Para este caso de estudio, compras de supermercado, cada transacción está formada por todos los productos que se compran a la vez, es decir, el vínculo de unión no es el cliente sino cada una de las “cestas de la compra”. Por ejemplo, la transacción con `id_compra == 14` está formada por 3 *items*.

Itemset (k = 3)	Transacciones	Soporte
{i1,i3,i5}	T2,T3	0.22
{i3,i4,i5}	T2,T9	0.22

El algoritmo finaliza cuando no hay más itemsets frecuentes.

Lectura de datos

Con la función `read.transactions()` se pueden leer directamente los datos de archivos tipo texto y almacenarlos en un objeto de tipo `transactions`, que es la estructura de almacenamiento que emplea arules. Esta función tiene los siguientes argumentos:

- `file`: nombre del archivo que se quiere leer.
- `format`: estructura en la que se encuentran almacenados los datos, “*basket*” si cada línea del archivo es una transacción completa, o “*single*” si cada línea representa un *item*.
- `sep`: tipo de separación de los campos.

- `cols`: si el formato es de tipo “*basket*”, un entero que indica la columna que contiene el identificador de las transacciones. Para el formato “*single*”, es un vector con los nombres (o posiciones) de las dos columnas que identifican las transacciones y los *items*, respectivamente.
- `rm.duplicates`: valor lógico indicando si se eliminan los *items* duplicados en una misma transacción. Por lo general, es conveniente eliminarlos, el interés suele estar en qué *items* ocurren de forma conjunta, no en qué cantidad.
- `quote`: carácter que se utiliza como comillas.
- `skip`: número de líneas que hay que saltar desde el comienzo del fichero.

Es importante recordar que los objetos de tipo `transactions` solo trabajan con información booleana, es decir, con la presencia o no de cada uno de los *items* en la transacción. Por lo tanto, si el set de datos contiene variables numéricas, estas deben de ser discretizadas en intervalos o niveles. Para discretizar una variable numérica, se puede emplear la función `discretize()` o bien otras alternativas como la función `case_when()` del paquete `dplyr`.

Los objetos `transactions`, se almacenan internamente como un tipo de matriz binaria. Se trata de una matriz de valores 0/1, con una fila por cada transacción, en este caso cada compra, y una columna por cada posible *item*, en este caso productos. La posición de la matriz (i,j) tiene el valor 1 si la transacción i contiene el *item* j .

```
# IMPORTACIÓN DIRECTA DE LOS DATOS A UN OBJETO TIPO TRANSACTION
# =====
library(arules)
transacciones <- read.transactions(file = "./datos/datos_groceries.csv",
                                   format = "single",
                                   sep = ",",
                                   header = TRUE,
                                   cols = c("id_compra", "item"),
                                   rm.duplicates = TRUE)

transacciones

## transactions in sparse format with
## 9835 transactions (rows) and
## 169 items (columns)
```

```
colnames(transacciones)[1:5]

## [1] "abrasive cleaner" "artif. sweetener" "baby cosmetics" "baby food"
## [5] "bags"

rownames(transacciones)[1:5]

## [1] "1" "10" "100" "1000" "1001"
```

También es posible convertir un objeto dataframe en uno de tipo transactions con la función `as(dataframe, "transactions")`. Para lograrlo, primero hay que convertir el dataframe en una lista en la que cada elemento contiene los *items* de una transacción. Este proceso puede ser muy lento si el dataframe tiene muchos registros, por lo que suele ser mejor crear un archivo de texto con los datos e importarlo mediante `read.transactions()`.

```
# CONVERSIÓN DE UN DATAFRAME A UN OBJETO TIPO TRANSACTION
# =====
# Se convierte el dataframe a una lista en la que cada elemento contiene
# los items de una transacción
datos_split <- split(x = datos$item, f = datos$id_compra)
transacciones <- as(datos_split, Class = "transactions")
transacciones

## transactions in sparse format with
## 9835 transactions (rows) and
## 169 items (columns)
```

Otra alternativa es convertir el dataframe en una matriz 0/1 en la que cada fila es una transacción y cada columna uno de los posibles *items*.

```
# CONVERSIÓN DE UNA MATRIZ A UN OBJETO TIPO TRANSACTION
# =====
datos_matriz <- datos %>%
  as.data.frame() %>%
  mutate(valor = 1) %>%
  spread(key = item, value = valor, fill = 0) %>%
  column_to_rownames(var = "id_compra") %>%
  as.matrix()
transacciones <- as(datos_matriz, Class = "transactions")
transacciones
```

```
## transactions in sparse format with  
## 9835 transactions (rows) and  
## 169 items (columns)
```

Exploración de items

Uno de los primeros análisis que conviene realizar cuando se trabaja con transacciones es explorar su contenido y tamaño, es decir, el número de *items* que las forman y cuáles son. Mediante la función `inspect()` se muestran los *items* que forman cada transacción.

```
inspect(transacciones[1:5])  
  
##      items                                transactionID  
## [1] {citrus fruit,  
##      margarine,  
##      ready soups,  
##      semi-finished bread}                            1  
## [2] {coffee,  
##      tropical fruit,  
##      yogurt}                                          2  
## [3] {whole milk}                                    3  
## [4] {cream cheese,  
##      meat spreads,  
##      pip fruit,  
##      yogurt}                                          4  
## [5] {condensed milk,  
##      long life bakery product,  
##      other vegetables,  
##      whole milk}                                    5
```

También es posible mostrar los resultados en formato de *dataframe* con la función `DATAFRAME()` o con `as(transacciones, "dataframe")`.

```
df_transacciones <- as(transacciones, Class = "data.frame")  
# Para que el tamaño de la tabla se ajuste mejor, se convierte el dataframe a tibble  
as.tibble(df_transacciones) %>% head()
```

items

<chr>

{citrus fruit,margarine,ready soups,semi-finished bread}

{coffee,tropical fruit,yogurt}

{whole milk}

{cream cheese,meat spreads,pip fruit,yogurt}

{condensed milk,long life bakery product,other vegetables,whole milk}

{abrasive cleaner,butter,rice,whole milk,yogurt}

6 rows | 1-1 of 2 columns

Para extraer el tamaño de cada transacción se emplea la función `size()`.

```
tamanyos <- size(transacciones)
summary(tamanyos)
```

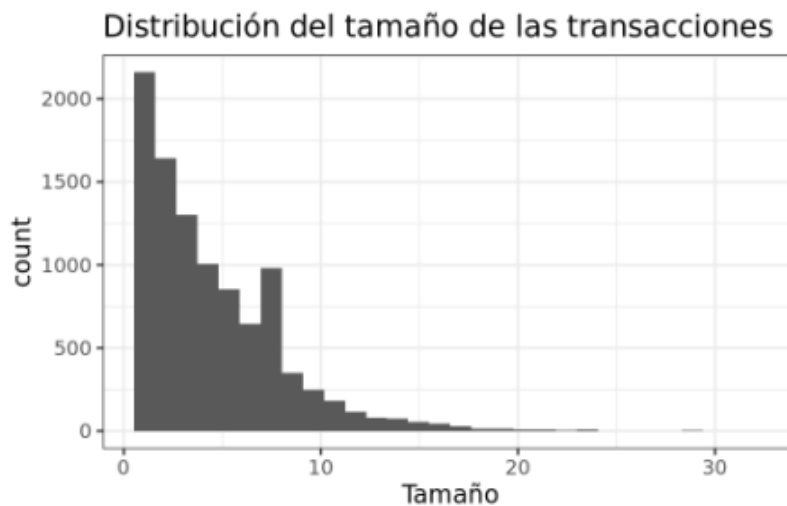
```
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##      1.000   2.000   3.000   4.409   6.000  32.000
```

```
quantile(tamanyos, probs = seq(0,1,0.1))
```

```
##      0%   10%   20%   30%   40%   50%   60%   70%   80%   90%  100%
##       1     1     1     2     3     3     4     5     7     9    32

data.frame(tamanyos) %>%
  ggplot(aes(x = tamanyos)) +
  geom_histogram() +
  labs(title = "Distribución del tamaño de las transacciones",
       x = "Tamaño") +
  theme_bw()
```

La gran mayoría de clientes compra entre 3 y 4 productos y el 90% de ellos compra como máximo 9. El siguiente análisis básico consiste en identificar cuáles son los *items* más frecuentes (los que tienen mayor soporte) dentro del conjunto de todas las transacciones. Con la función `itemFrequency()` se puede extraer esta información de un objeto tipo `transactions`.



El nombre de esta función puede causar confusión. Por “frecuencia” se hace referencia al soporte de cada *item*, que es la fracción de transacciones que contienen dicho *item* respecto al total de todas las transacciones. Esto es distinto a la frecuencia de un *item* respecto al total de *items*, de ahí que la suma de todos los soportes no sea 1.

```
frecuencia_items <- itemFrequency(x = transacciones, type = "relative")  
frecuencia_items %>% sort(decreasing = TRUE) %>% head(5)
```

```
##      whole milk other vegetables      rolls/buns      soda  
##      0.2555160      0.1934926      0.1839349      0.1743772  
##      yogurt  
##      0.1395018
```

Si se indica el argumento `type = "absolute"`, la función `itemFrequency()` devuelve el número de transacciones en las que aparece cada *item*.

```
frecuencia_items <- itemFrequency(x = transacciones, type = "absolute")  
frecuencia_items %>% sort(decreasing = TRUE) %>% head(5)
```

```
##      whole milk other vegetables      rolls/buns      soda  
##      2513              1903          1809          1715  
##      yogurt  
##      1372
```

El listado anterior muestra que los 5 productos que más se venden son: *whole milk*, *other vegetables*, *rolls/buns* y *soda*. Es muy importante estudiar cómo se distribuye el soporte de los *items* individuales en un conjunto de transacciones antes identificar *itemsets* frecuentes o crear reglas de asociación, ya que, dependiendo del caso, tendrá sentido emplear un límite de soporte u otro. Por lo general, cuando el número de posibles *items* es muy grande (varios miles) prácticamente todos los artículos son raros, por lo que los soportes son muy bajos.

Itemsets

Con la función `apriori()` se puede aplicar el algoritmo *Apriori* a un objeto de tipo `transactions` y extraer tanto *itemsets* frecuentes como reglas de asociación que superen un determinado soporte y confianza. Los argumentos de esta función son:

- `data`: un objeto del tipo `transactions` o un objeto que pueda ser convertido a tipo `transactions`, por ejemplo un *dataframe* o una matriz binaria.
- `parameter`: lista en la que se indican los parámetros del algoritmo.
- `support`: soporte mínimo que debe tener un *itemset* para ser considerado frecuente. Por defecto es 0.1.
- `minlen`: número mínimo de *items* que debe tener un *itemset* para ser incluido en los resultados. Por defecto 1.
- `maxlen`: número máximo de *items* que puede tener un *itemset* para ser incluido en los resultados. Por defecto 10.

- target: tipo de resultados que debe de generar el algoritmo, pueden ser “frequent itemsets”, “maximally frequent itemsets”, “closed frequent itemsets”, “rules” o “hyperedgesets”.
- confidence: confianza mínima que debe de tener una regla para ser incluida en los resultados. Por defecto 0.8.
- maxtime: tiempo máximo que puede estar el algoritmo buscando subsets. Por defecto 5 segundos.
- appearance: lista que permite definir patrones para restringir el espacio de búsqueda, por ejemplo, especificando qué *items* pueden o no pueden aparecer.
- control: lista que permite modificar aspectos internos de algoritmo como la ordenación de los *itemsets*, si se construye un árbol con las transacciones, aspectos relacionados con el uso de memoria, etc.

Se procede a extraer aquellos *itemsets*, incluidos los formados por un único *item*, que hayan sido comprados al menos 30 veces. En un caso real, este valor sería excesivamente bajo si se tiene en cuenta la cantidad total de transacciones, sin embargo, se emplea 30 para que en los resultados aparezcan un número suficiente de *itemsets* y reglas de asociación que permitan mostrar las posibilidades de análisis que ofrece el paquete arules.

```
soporte <- 30 / dim(transacciones)[1]
itemsets <- apriori(data = transacciones,
                    parameter = list(support = soporte,
                                     minlen = 1,
                                     maxlen = 20,
                                     target = "frequent itemset"))
```

```
## Apriori
##
## Parameter specification:
## confidence minval smax arem aval originalSupport maxtime support minlen
##      NA  0.1  1 none FALSE      TRUE    5 0.00305033  1
## maxlen      target ext
##      20 frequent itemsets TRUE
##
## Algorithmic control:
## filter tree heap memopt load sort verbose
```



```
## 0.1 TRUE TRUE FALSE TRUE 2 TRUE
##
## Absolute minimum support count: 30
##
## set item appearances ...[0 item(s)] done [0.00s].
## set transactions ...[169 item(s), 9835 transaction(s)] done [0.00s].
## sorting and recoding items ... [136 item(s)] done [0.00s].
## creating transaction tree ... done [0.00s].
## checking subsets of size 1 2 3 4 5 done [0.00s].
## sorting transactions ... done [0.00s].
## writing ... [2226 set(s)] done [0.00s].
## creating S4 object ... done [0.00s].
```

```
summary(itemsets)
```

```
## set of 2226 itemsets
##
## most frequent items:
##   whole milk other vegetables      yogurt root vegetables
##      556      468      316      251
##   rolls/buns      (Other)
##      241      3536
##
## element (itemset/transaction) length distribution:sizes
## 1 2 3 4 5
## 136 1140 850 98 2
##
##   Min. 1st Qu.  Median    Mean 3rd Qu.  Max.
## 1.000 2.000 2.000 2.412 3.000 5.000
##
## summary of quality measures:
##   support      transIdenticalToItemsets      count
## Min. :0.003050 Min. :0.0000000 Min. : 30.00
## 1st Qu.:0.003660 1st Qu.:0.0000000 1st Qu.: 36.00
## Median :0.004779 Median :0.0000000 Median : 47.00
## Mean :0.007879 Mean :0.0001613 Mean : 77.49
## 3rd Qu.:0.007219 3rd Qu.:0.0001017 3rd Qu.: 71.00
## Max. :0.255516 Max. :0.0265379 Max. :2513.00
##
## includes transaction ID lists: FALSE
##
## mining info:
##   data ntransactions      support confidence
## transacciones      9835 0.00305033      1
```

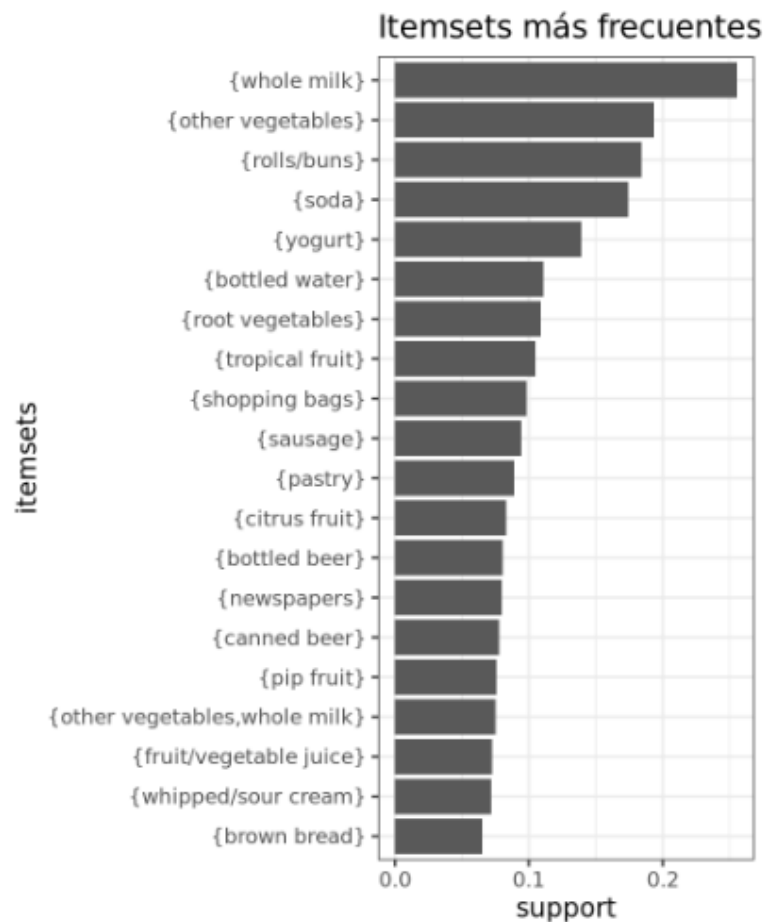
Se han encontrado un total de 2226 *itemsets* frecuentes que superan el soporte mínimo de 0.003908286, la mayoría de ellos (1140) formados por dos *items*. En el siguiente listado se muestran los 20 *itemsets* con mayor soporte que,

como cabe esperar, son los formados por *items* individuales (los *itemsets* de menor tamaño).

```
# Se muestran los top 20 itemsets de mayor a menor soporte
top_20_itemsets <- sort(itemsets, by = "support", decreasing = TRUE)[1:20]
inspect(top_20_itemsets)
```

##	items	support	transIdenticalToItemsets	count
## [1]	{whole milk}	0.25551601	0.0125063549	2513
## [2]	{other vegetables}	0.19349263	0.0063040163	1903
## [3]	{rolls/buns}	0.18393493	0.0111845450	1809
## [4]	{soda}	0.17437722	0.0162684291	1715
## [5]	{yogurt}	0.13950178	0.0040671073	1372
## [6]	{bottled water}	0.11052364	0.0068124047	1087
## [7]	{root vegetables}	0.10899847	0.0025419420	1072
## [8]	{tropical fruit}	0.10493137	0.0023385867	1032
## [9]	{shopping bags}	0.09852567	0.0048805287	969
## [10]	{sausage}	0.09395018	0.0024402644	924
## [11]	{pastry}	0.08896797	0.0037620742	875
## [12]	{citrus fruit}	0.08276563	0.0017285206	814
## [13]	{bottled beer}	0.08052872	0.0124046772	792
## [14]	{newspapers}	0.07981698	0.0055922725	785
## [15]	{canned beer}	0.07768175	0.0265378749	764
## [16]	{pip fruit}	0.07564820	0.0028469751	744
## [17]	{other vegetables,whole milk}	0.07483477	0.0006100661	736
## [18]	{fruit/vegetable juice}	0.07229283	0.0022369090	711
## [19]	{whipped/sour cream}	0.07168277	0.0026436197	705
## [20]	{brown bread}	0.06487036	0.0021352313	638

```
# Para representarlos con ggplot se convierte a dataframe
as(top_20_itemsets, Class = "data.frame") %>%
  ggplot(aes(x = reorder(items, support), y = support)) +
  geom_col() +
  coord_flip() +
  labs(title = "Itemsets más frecuentes", x = "itemsets") +
  theme_bw()
```



Si se quieren excluir del análisis los *itemsets* formados únicamente por un solo *item*, se puede, o bien aplicar de nuevo la función `apriori()` especificando `minlen = 2`, o filtrar los resultados con la función `size()`.

```
# Se muestran los 20 itemsets más frecuentes formados por más de un item.
inspect(sort(itemsets[size(itemsets) > 1], decreasing = TRUE)[1:20])
```

##	items	support	transIdenticalToItem
## [1]	{other vegetables, whole milk}	0.07483477	0.0006100661
## [2]	{rolls/buns, whole milk}	0.05663447	0.0013218099
## [3]	{whole milk, yogurt}	0.05602440	0.0008134215
## [4]	{root vegetables, whole milk}	0.04890696	0.0003050330
## [5]	{other vegetables, root vegetables}	0.04738180	0.0006100661
## [6]	{other vegetables, yogurt}	0.04341637	0.0004067107
## [7]	{other vegetables, rolls/buns}	0.04260295	0.0008134215
## [8]	{tropical fruit, whole milk}	0.04229792	0.0008134215
## [9]	{soda, whole milk}	0.04006101	0.0007117438
## [10]	{rolls/buns, soda}	0.03833249	0.0023385867
## [11]	{other vegetables, tropical fruit}	0.03589222	0.0005083884

```
## [12] {bottled water,whole milk}      0.03436706 0.0007117438
## [13] {rolls/buns,yogurt}             0.03436706 0.0010167768
## [14] {pastry,whole milk}             0.03324860 0.0012201322
## [15] {other vegetables,soda}         0.03274021 0.0001016777
## [16] {whipped/sour cream,whole milk} 0.03223183 0.0008134215
## [17] {rolls/buns,sausage}           0.03060498 0.0013218099
## [18] {citrus fruit,whole milk}       0.03050330 0.0003050330
## [19] {pip fruit,whole milk}         0.03009659 0.0004067107
## [20] {domestic eggs,whole milk}     0.02999492 0.0003050330
## count
## [1] 736
## [2] 557
## [3] 551
## [4] 481
## [5] 466
## [6] 427
## [7] 419
## [8] 416
## [9] 394
## [10] 377
## [11] 353
## [12] 338
## [13] 338
## [14] 327
## [15] 322
## [16] 317
## [17] 301
## [18] 300
## [19] 296
## [20] 295
```

Filtrado de itemsets

Una vez que los *itemsets* frecuentes han sido identificados mediante el algoritmo *Apriori*, pueden ser filtrados con la función `subset()`. Esta función recibe dos argumentos: un objeto *itemset* o *rules* y una condición lógica que tienen que cumplir las reglas/*itemsets* para ser seleccionados. La siguiente tabla muestra los operadores permitidos:

Operador	Significado
&	AND
%in%	contiene cualquier de los siguientes elementos
%ain%	contiene todos de los siguientes elementos
%pin%	contiene parcialmente los siguientes elementos

Como esta función tiene el mismo nombre que una función del paquete básico de R, para evitar errores, es conveniente especificar el paquete donde se encuentra. Se procede a identificar aquellos *itemsets* frecuentes que contienen el *item newspapers*.

```
itemsets_filtrado <- arules::subset(itemsets,  
                                   subset = items %in% "newspapers")  
itemsets_filtrado
```

```
## set of 80 itemsets
```

```
# Se muestran 10 de ellos  
inspect(itemsets_filtrado[1:10])
```

Se repite el proceso pero, esta vez, con aquellos *itemsets* que contienen *newspapers* y *whole milk*.

```
itemsets_filtrado <- arules::subset(itemsets,  
                                   subset = items %ain% c("newspapers", "  
whole milk"))  
itemsets_filtrado
```

```
# Se muestran 10 de ellos  
inspect(itemsets_filtrado[1:10])
```

Puede observarse que muchos *itemsets* están a su vez contenidos en *itemsets* de orden superior, es decir, existen *itemsets* que son subsets de otros. Para identificar cuáles son, o cuales no lo son, se puede emplear la función `is.subset()`. Encontrar los *itemsets* que son subsets de otros *itemsets* implica comparar todos los pares de *itemsets* y determinar si uno está contenido en el otro. La función `is.subset()` realiza comparaciones entre dos conjuntos de *itemsets* y

devuelve una matriz lógica que determina si el *itemset* de la fila está contenido en cada *itemset* de las columnas.

```
# Para encontrar los subsets dentro de un conjunto de itemsets, se compara el  
# conjunto de itemsets con sí mismo.  
subsets <- is.subset(x = itemsets, y = itemsets, sparse = FALSE)
```

Para conocer el total de *itemsets* que son subsets de otros *itemsets* se cuenta el número total de TRUE en la matriz resultante.

```
# La suma de una matriz lógica devuelve el número de TRUEs  
sum(subsets)
```

```
## [1] 11038
```

Reglas de asociación

Para crear las reglas de asociación se sigue el mismo proceso que para obtener *itemsets* frecuentes pero, además de especificar un soporte mínimo, se tiene que establecer una confianza mínima para que una regla se incluya en los resultados. En este caso, se emplea una confianza mínima del 70%.

```
soporte <- 30 / dim(transacciones)[1]  
reglas <- apriori(data = transacciones,  
                  parameter = list(support = soporte,  
                                   confidence = 0.70,  
                                   # Se especifica que se creen reglas  
                                   target = "rules"))
```

```
## Apriori
##
## Parameter specification:
## confidence minval smax arem aval originalSupport maxtime support minlen
##      0.7      0.1      1 none FALSE                TRUE      5 0.00305033      1
## maxlen target ext
##      10 rules TRUE
##
## Algorithmic control:
## filter tree heap memopt load sort verbose
##      0.1 TRUE TRUE  FALSE TRUE      2      TRUE
##
## Absolute minimum support count: 30
##
## set item appearances ...[0 item(s)] done [0.00s].
## set transactions ...[169 item(s), 9835 transaction(s)] done [0.00s].
## sorting and recoding items ... [136 item(s)] done [0.00s].
## creating transaction tree ... done [0.00s].
## checking subsets of size 1 2 3 4 5 done [0.00s].
## writing ... [19 rule(s)] done [0.00s].
## creating S4 object ... done [0.00s].
```

summary(reglas)

```
## set of 19 rules
##
## rule length distribution (lhs + rhs):sizes
## 3 4 5
## 7 9 3
##
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##      3.000  3.000  4.000  3.789  4.000  5.000
##
## summary of quality measures:
##      support      confidence      coverage      lift
##      Min.   :0.003050  Min.   :0.7000  Min.   :0.003559  Min.   :2.740
##      1st Qu.:0.003203  1st Qu.:0.7047  1st Qu.:0.004525  1st Qu.:2.758
##      Median :0.003559  Median :0.7164  Median :0.004982  Median :2.804
##      Mean   :0.003767  Mean   :0.7373  Mean   :0.005143  Mean   :3.044
##      3rd Qu.:0.004169  3rd Qu.:0.7500  3rd Qu.:0.005592  3rd Qu.:2.984
##      Max.   :0.005694  Max.   :0.8857  Max.   :0.008134  Max.   :4.578
##      count
##      Min.   :30.00
##      1st Qu.:31.50
##      Median :35.00
##      Mean   :37.05
##      3rd Qu.:41.00
##      Max.   :56.00
##
## mining info:
##      data ntransactions support confidence
## transacciones      9835 0.00305033      0.7
```

Se han identificado un total de 19 reglas, la mayoría de ellas formadas por 4 *items* en el antecedente (parte izquierda de la regla). A continuación, se muestran 6 de estas.

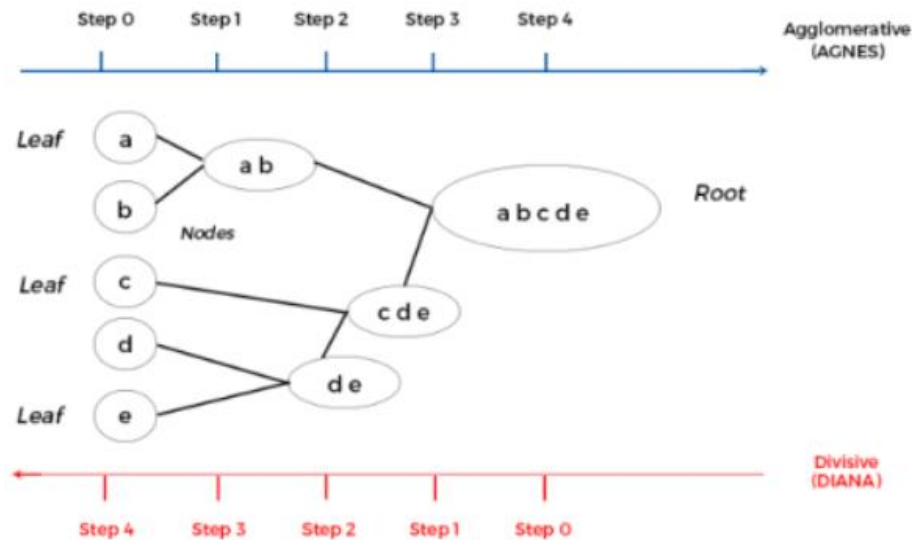
```
inspect(sort(x = reglas, decreasing = TRUE, by = "confidence"))
```

##	lhs	rhs	support	confidence	coverage	lift	count
## [1]	{citrus fruit,						
##	root vegetables,						
##	tropical fruit,						
##	whole milk}	=> {other vegetables}	0.003152008	0.8857143	0.003558719	4.577509	31
## [2]	{butter,						
##	root vegetables,						
##	yogurt}	=> {whole milk}	0.003050330	0.7894737	0.003863752	3.089723	30
## [3]	{citrus fruit,						
##	root vegetables,						
##	tropical fruit}	=> {other vegetables}	0.004473818	0.7857143	0.005693950	4.060694	44
## [4]	{brown bread,						
##	other vegetables,						
##	root vegetables}	=> {whole milk}	0.003152008	0.7750000	0.004067107	3.033078	31
## [5]	{butter,						
##	onions}	=> {whole milk}	0.003050330	0.7500000	0.004067107	2.935237	30
## [6]	{curd,						
##	tropical fruit,						
##	yogurt}	=> {whole milk}	0.003965430	0.7500000	0.005287239	2.935237	39

2.2 Clustering jerárquico

En Clustering Jerárquico, los clusters se crean de manera que tengan un orden predeterminado, es decir, una jerarquía. Por ejemplo, considere la jerarquía de conceptos de una biblioteca. Una biblioteca tiene muchas secciones, cada sección tendría muchos libros, y los libros se agruparían según su tema, digamos. Esto forma una jerarquía que puede crearse de arriba a abajo o viceversa (Jiménez, 2018). Existen dos métodos, el divisivo y el aglomerativo.

En el **método divisivo**, suponemos que todas las observaciones pertenecen a un único grupo y luego dividimos el clúster en dos grupos menos similares. Esto se repite recursivamente en cada grupo hasta que haya un grupo para cada observación. Esta técnica también se llama DIANA, que es un acrónimo de Análisis Divisivo. El **método aglomerativo**, también se lo conoce como aglomeración aglomerativa jerárquica (HAC) o AGNES (acrónimo de aglomeración de anidación).



En este método, cada observación se asigna a su propio clúster. Luego, se calcula la similitud (o distancia) entre cada uno de los clusters y los dos clusters más similares se fusionan en uno. Finalmente, los pasos 2 y 3 se repiten hasta que solo quede un grupo. HAC representa la mayoría de los algoritmos de agrupación jerárquica, mientras que los métodos divisivos rara vez se utilizan.



Recuerde que: El método divisivo es bueno para identificar clusters grandes, mientras que el método aglomerativo es bueno para identificar clusters pequeños (Jiménez, 2018).

Algoritmo HAC

Dado un conjunto de N elementos a agrupar, y una matriz de distancia $N \times N$ (o similitud), el proceso básico del agrupamiento jerárquico de Johnson, 1967, citado en Jiménez, (2018) es:

1. signe cada elemento a su propio clúster, de modo que, si tiene N elementos, ahora tiene N clusters, cada uno con un solo elemento. Deje que las distancias (similitudes) entre los conglomerados sean iguales a las distancias (similitudes) entre los elementos que contienen.
2. Encuentre el par más cercano (más similar) de clústeres y combínelos en un único clúster, de modo que ahora tenga un clúster menos.

3. Calcule distancias (similitudes) entre el nuevo clúster y cada uno de los clusters antiguos.
4. Repita los pasos 2 y 3 hasta que todos los elementos estén agrupados en un solo grupo de tamaño N.

En los Pasos 2 y 3 de este documento, el algoritmo habla sobre la búsqueda de similitud entre los clusters. Entonces, antes de realizar cualquier agrupamiento, se requiere determinar la matriz de distancia que especifica la distancia entre cada punto de datos usando alguna función de distancia (Euclidiana, Manhattan, Minkowski, etc.). Luego, la matriz se actualiza para especificar la distancia entre los diferentes clústeres que se forman como resultado de la fusión. Pero, ¿cómo medimos la distancia (o similitud) entre dos grupos de observaciones? Para esto, tenemos tres métodos diferentes como se describe a continuación. Estos métodos difieren en cómo se mide la distancia entre cada grupo.



Figura 8. Criterios de enlace del algoritmo jerárquico (Párraga-Álava, 2020).

En relación a la figura 8, al contar con la matriz de distancia entre cada par de objeto, podemos aplicar el algoritmo de clustering jerárquico. Este algoritmo requiere algún criterio de enlace para la formación de grupos (Párraga-Álava, 2020).

El método de enlace completo también se llama diámetro o método máximo. En este método, consideramos la similitud del par más lejano. Es decir, la distancia entre un clúster y otro clúster se considera igual a la distancia más larga desde cualquier miembro de un clúster a cualquier miembro del otro clúster. Tiende a producir clusters más compactos. Una desventaja de este método es que los valores atípicos pueden provocar la fusión de grupos cercanos más tarde de lo que es óptimo.

En el método de enlace promedio, tomamos la distancia entre un clúster y otro clúster para que sea igual a la distancia promedio desde cualquier miembro de un clúster a cualquier miembro del otro clúster.

El análisis de agrupamiento es una técnica para agrupar objetos (registros) similares

en varios grupos en función de las características (variables) de cada individuo (Párraga-Álava, 2020). Para este análisis se requiere:

1. Características de objetos a agrupar.
2. Distancia/similitud entre objetos.
3. Método de agrupamiento.

Las métricas de distancia/similitud sirven para computar la “distancia” (menor) o “similitud” (mayor) entre dos objetos, basados en sus características. Entre las métricas comúnmente utilizadas están:

- Euclídea
- Manhattan
- Hamming
- Levenshtein
- Similitud de Pearson

La información de las distancias/similitud entre objetos se organizan en una matriz:

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
1	0	12.789	14.029	5.554	7.668	12.698	12.352	11.586	12.95	14.271	13.903	19.07	12.96	11.324	14.304	4.03	17.932
2	12.789	0	17.132	4.879	1.935	0.15	1.074	0.871	0.435	0.131	0.129	14.262	0.47	0.215	0.13	9.257	16.12
3	14.029	17.132	0	6.121	16.162	14.541	18.139	16.131	18.293	19.235	18.702	15.44	18.399	18.498	19.258	24.689	26.439
4	5.554	4.879	6.121	0	5.423	3.93	6.843	5.551	6.255	6.168	5.933	14.802	6.353	5.121	6.17	7.563	21.002
5	7.668	1.935	16.162	5.423	0	2.277	1.02	1.171	1.375	2.263	2.132	10.085	1.342	1.52	2.288	7.139	8.835
6	12.698	0.15	14.541	3.93	2.277	0	1.501	1.124	0.811	0.468	0.441	13.015	0.859	0.63	0.466	10.039	16.845
7	12.352	1.074	18.139	6.843	1.02	1.501	0	0.101	0.172	0.836	0.733	14.189	0.15	0.777	0.857	11.003	9.16
8	11.586	0.871	16.131	5.551	1.171	1.124	0.101	0	0.167	0.729	0.605	14.435	0.162	0.65	0.746	10.696	9.95
9	12.95	0.435	18.293	6.255	1.375	0.811	0.172	0.167	0	0.251	0.195	14.886	0.001	0.308	0.262	10.48	11.836
10	14.271	0.131	19.235	6.168	2.263	0.468	0.836	0.729	0.251	0	0.007	16.188	0.278	0.204	0	10.334	15.532
11	13.903	0.129	18.702	5.933	2.132	0.441	0.733	0.605	0.195	0.007	0	16.007	0.221	0.184	0.008	10.288	15.062
12	19.07	14.262	15.44	14.802	10.085	13.015	14.189	14.435	14.886	16.188	16.007	0	14.831	15.946	16.222	22.93	19.505
13	12.96	0.47	18.399	6.353	1.342	0.859	0.15	0.162	0.001	0.278	0.221	14.831	0	0.331	0.29	10.516	11.654
14	11.324	0.215	18.498	5.121	1.52	0.63	0.777	0.65	0.308	0.204	0.184	15.946	0.331	0	0.208	7.796	14.602
15	14.304	0.13	19.258	6.17	2.288	0.466	0.857	0.746	0.262	0	0.008	16.222	0.29	0.208	0	10.337	15.619
16	4.03	9.257	24.689	7.563	7.139	10.039	11.003	10.696	10.48	10.334	10.288	22.93	10.516	7.796	10.337	0	25.143
17	17.932	16.12	26.439	21.002	8.835	16.845	9.16	9.95	11.836	15.532	15.062	19.505	11.654	14.602	15.619	25.143	0

1. Argentina, 2. Bolivia, 3. Brasil, 4. Chile, 5. Colombia, 6. Costa Rica, 7. Cuba, 8. Ecuador, 9. El Salvador, 10. Guatemala, 11. Honduras, 12. México, 13. Nicaragua, 14. Paraguay, 15. Perú, 16. Uruguay, 17. Venezuela.

Figura 9. Distancia/similitud (Párraga-Álava, 2020).

En la figura 10, se puede analizar los clientes más cercanos que van a la misma pizzería. En este caso, se ilustra tres grupos de 2 clientes que acuden a la misma pizzería.



Figura 10. Análisis del agrupamiento jerárquico (Párraga-Álava, 2020).

Finalmente, se presenta la organización de los grupos mediante el Dendograma de la figura 11.

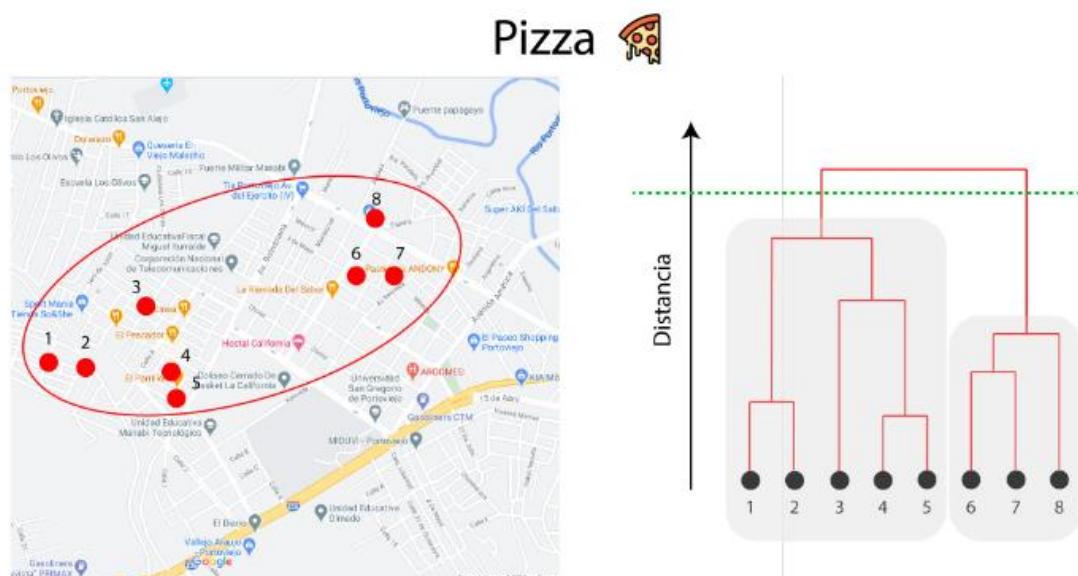


Figura 11. Dendograma de agrupamiento jerárquico (Párraga-Álava, 2020).



Recuerde que: El agrupamiento jerárquico, también conocido como análisis de conglomerados jerárquicos, es un algoritmo que agrupa objetos similares en grupos llamados conglomerados. El punto final es un conjunto de clústeres, donde cada clúster es distinto de los demás y los objetos dentro de cada clúster son similares entre sí.

EJEMPLO DE CLUSTERING JERÁRQUICO

Se utilizará en este ejemplo, los paquetes de la figura 12, tidyverse, magrittr, dplyr, amap, dendextend, factoextra, purrr.

```
if(!require(tidyverse)) {install.packages("tidyverse")}
if(!require(magrittr)) {install.packages("magrittr")}
if(!require(dplyr)) {install.packages("dplyr")}
if(!require(ama)) {install.packages("ama")}
if(!require(dendextend)) {install.packages("dendextend")}
if(!require(factoextra)) {install.packages("factoextra")}
if(!require(purrr)) {install.packages("purrr")}
```

Figura 12. Paquetes requeridos (Párraga-Álava, 2020).

Una vez que se ha instalado los paquetes, se procede a cargar los mismos mediante:

```
library("tidyverse")
library("magrittr")
library("dplyr")
library("amap")
library("dendextend")
library("factoextra")
library("purrr")
```

Figura 13. Carga de paquetes (Párraga-Álava, 2020).

Los datos a utilizar provienen de <https://github.com/jorgeklz/curso-utm/blob/main/proteina.csv>. El conjunto de datos proteina contiene datos del consumo proteínico de 24 países de Europa almacenados como ingestas de proteínas (en porcentaje) de nueve fuentes principales de alimentos (Párraga-Álava, 2020).

Cargamos los datos:

<https://raw.githubusercontent.com/jorgeklz/curso-utm/main/proteina.csv>

```
proteina <- read_csv("https://raw.githubusercontent.com/jorgeklz/curso-utm/main/proteina.csv")
proteina
```

```
## # A tibble: 24 x 10
##   pais carne_roja carne_blanca huevos leche pescado cereales almidon nueces
##   <chr>      <dbl>      <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl>
## 1 Alba~      10.1        1.4    0.5  8.9    0.2   42.3    0.6    5.5
## 2 Aust~       8.9        14     4.3 19.9    2.1    28     3.6    1.3
## 3 Bélg~      13.5        9.3    4.1 17.5    4.5   26.6    5.7    2.1
## 4 Bulg~       7.8         6     1.6  8.3    1.2   56.7    1.1    3.7
## 5 Repú~       9.7       11.4    2.8 12.5     2    34.3     5     1.1
## 6 Dina~      10.6       10.8    3.7 25     9.9   21.9    4.8    0.7
## 7 Alem~       8.4       11.6    3.7 11.1    5.4   24.6    6.5    0.8
## 8 Finl~       9.5        4.9    2.7 33.7    5.8   26.3    5.1     1
## 9 Fran~      18         9.9    3.3 19.5    5.7   28.1    4.8    2.4
## 10 Grec~     10.2         3     2.8 17.6    5.9   41.7    2.2    7.8
## # ... with 14 more rows, and 1 more variable: frutas_vegetales <dbl>
```

Figura 14. Carga de los datos (Párraga-Álava, 2020).

Los datos en las columnas deben estar estandarizados o escalados. Este proceso hace que cada variable tenga una media cero y una desviación estándar uno. Esto se hace para que una unidad en cada variable represente el mismo grado de diferencia (Párraga-Álava, 2020). Usamos la función **scale** de R-base, pero únicamente a variables numéricas.

```
proteina_escala <- proteina %>%
  mutate_if(is.numeric, scale)
proteina_escala
```

```
## # A tibble: 24 x 10
##   pais carne_roja[,1] carne_blanca[,1] huevos[,1] leche[,1] pescado[,1]
##   <chr>          <dbl>          <dbl>          <dbl>          <dbl>          <dbl>
## 1 Alba~         0.0992         -1.73         -2.14         -1.12         -1.19
## 2 Aust~        -0.253          1.73          1.27          0.394        -0.640
## 3 Bélg~         1.10          0.438         1.09          0.0632        0.0516
## 4 Bulg~        -0.577         -0.468        -1.16         -1.21        -0.899
## 5 Repú~        -0.0184         1.01        -0.0785        -0.626        -0.669
## 6 Dina~         0.246         0.850         0.729         1.10         1.61
## 7 Alem~        -0.400         1.07         0.729        -0.820         0.311
## 8 Finl~        -0.0771        -0.770        -0.168         2.30         0.426
## 9 Fran~         2.42          0.603         0.370         0.339         0.397
## 10 Grec~         0.129         -1.29        -0.0785         0.0770         0.455
## # ... with 14 more rows, and 4 more variables: cereales[,1] <dbl>,
## #   almidon[,1] <dbl>, nueces[,1] <dbl>, frutas_vegetales[,1] <dbl>
```

Figura 15. Estandarización de los datos (Párraga-Álava, 2020).

Medidas de distancias/similitud.

Luego de contar con los datos, se debe identificar la distancia o similitud entre objetos, por ello procedemos a calcular la matriz de distancia entre cada una de las 24 instancias (países) de la base de datos (Párraga-Álava, 2020). Usamos la función **Dist()** del paquete de amap en figura 16.

```
# Euclidean
distancia_euclidean <- proteina_escala %>% select_if(is.numeric) %>%
  Dist(method = "euclidean", diag = FALSE, upper = TRUE) %>% as.matrix()
distancia_euclidean
```

```
# Manhattan
distancia_manhattan <- proteina_escala %>% select_if(is.numeric) %>%
  Dist(method = "manhattan", diag = FALSE, upper = TRUE) %>% as.matrix()
distancia_manhattan
```

Figura 16. Estandarización de los datos (Párraga-Álava, 2020).

Con los criterios de enlace, explicados en la figura 8, se procede a crearlos. Aplicamos el algoritmo jerárquico con la función **hclust()** del paquete **stats** de R-base.

```
hc.simple <- hclust(dist(distancia_euclidea), method = "single")  
hc.completo <- hclust(dist(distancia_euclidea), method = "complete")  
hc.promedio <- hclust(dist(distancia_euclidea), method = "average")
```

```
##  
## Call:  
## hclust(d = dist(distancia_euclidea), method = "single")  
##  
## Cluster method   : single  
## Distance         : euclidean  
## Number of objects: 24
```

Figura 17. Criterios de enlace con distancia euclídea (Párraga-Álava, 2020).

Note que creamos un objeto para cada criterio de enlace, y en todos estos casos usando la distancia euclídea.

Para ver el resultado del agrupamiento usamos la función `plot()`.

```
# visualiza dendrograma  
plot(hc.simple, main="Agrupamiento con criterio SIMPLE",  
      xlab="Países", ylab="Distancia", cex=1)  
  
# cambiamos las etiquetas de los objetos  
hc.simple$labels <- proteina_escalas$pais[as.numeric(hc.simple$labels)]  
  
# volvemos a plotear  
plot(hc.simple, main="Agrupamiento con criterio SIMPLE",  
      xlab="Países", ylab="Distancia", cex=1)
```

Figura 18. Mostrar resultado de agrupamiento (Párraga-Álava, 2020).

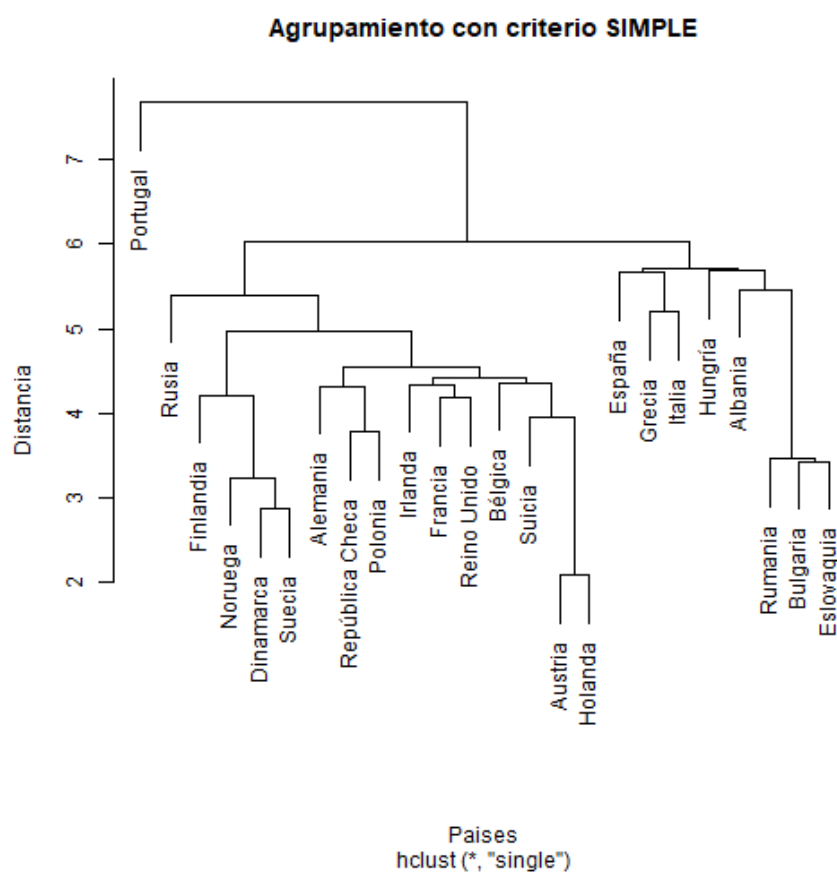
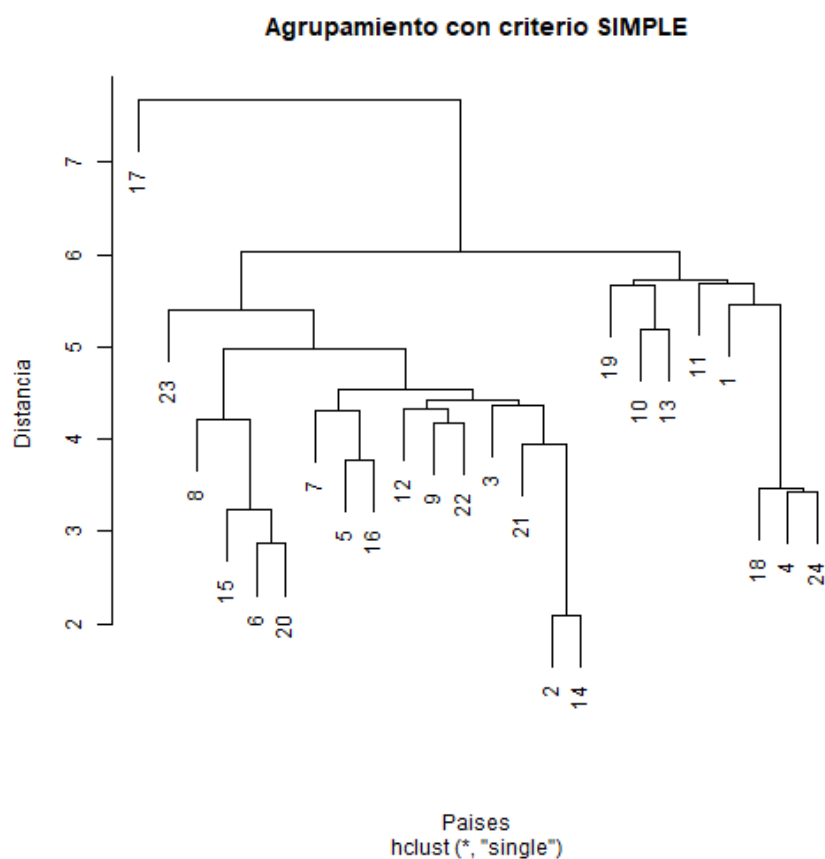


Figura 19. Agrupamiento con criterio simple (Párraga-Álava, 2020).

Ahora indicamos el punto de corte para la formación de grupos con la función **cutree()**, también del paquete **stats**.

```
# corte como la altura (distancia)
grupo <- cutree(hc.simple, h=6.2)

# genera el agrupamiento en modo matriz
objeto <- proteina_escalas$pais
agrupamiento <- as_tibble(cbind(objeto, grupo))
agrupamiento
```

```
## # A tibble: 24 x 2
##   objeto      grupo
##   <chr>      <chr>
## 1 Albania      1
## 2 Austria      1
## 3 Bélgica      1
## 4 Bulgaria     1
## 5 República Checa 1
## 6 Dinamarca    1
## 7 Alemania     1
## 8 Finlandia    1
## 9 Francia      1
## 10 Grecia      1
## # ... with 14 more rows
```

Figura 20. Punto de corte, criterio altura (distancia) (Párraga-Álava, 2020).

También es posible, indicar el número de grupos deseados, mediante $K = 3$.

```
# corte número de grupos
grupo <- cutree(hc.simple, k=3)

# genera el agrupamiento en modo matriz
objeto <- proteina_escalas$pais
agrupamiento <- as_tibble(cbind(objeto, grupo))
agrupamiento
```

```
## # A tibble: 24 x 2
##   objeto      grupo
##   <chr>      <chr>
## 1 Albania      1
## 2 Austria      2
## 3 Bélgica      2
## 4 Bulgaria     1
## 5 República Checa 2
## 6 Dinamarca    2
## 7 Alemania     2
## 8 Finlandia    2
## 9 Francia      2
## 10 Grecia      1
## # ... with 14 more rows
```

Figura 21. Punto de corte, criterio por número de grupos (Párraga-Álava, 2020).

Es posible mejorar la visualización de los grupos en el dendrograma, para ello usamos la funciones **color_branches()** y **color_labels()** del paquete **dendextend**.

```
# Colores de las ramas
hc.simple <- color_branches(hc.simple, k = 3)

# Colores de las etiquetas (objetos)
hc.simple <- color_labels(hc.simple, k = 3)

# Tamano de fuente de etiquetas
hc.simple <- set(hc.simple, "labels_cex", 1)

# plot
plot(hc.simple, main="Agrupamiento con criterio SIMPLE",
      xlab="Países", ylab="Distancia", cex=1)
```

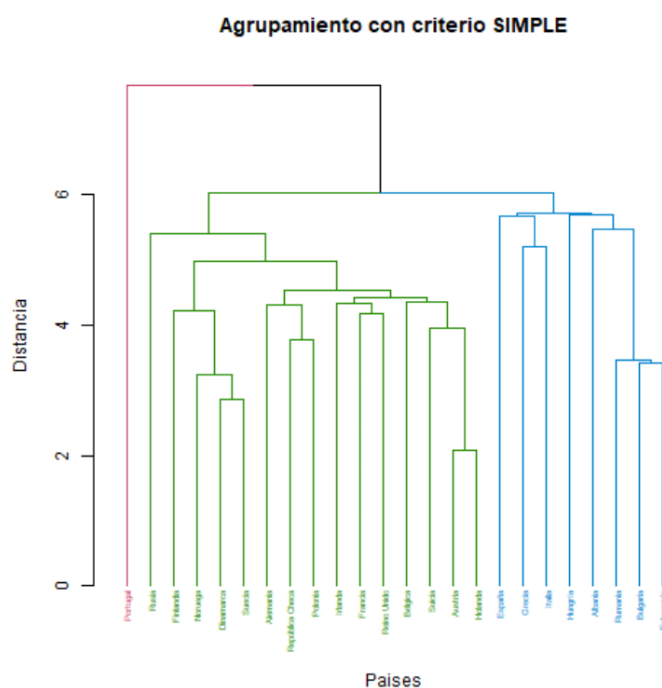


Figura 22. Visualización del dendrograma (Párraga-Álava, 2020).

Finalmente podríamos evaluar el rendimiento de los tres criterios de enlace con diferentes valores de corte. Usamos la función **hcut()** del paquete **factoextra** y la función **map_df** del paquete **purrr**, como se presenta en la figura 23.

```
# evalúa valores de números de grupos (k) de 3 a 10 para cada criterio de
# enlace de clustering y reporta una tabla con las siluetas promedio que es
# es una medida de calidad de agrupamiento.

resultados <- map_df(c(3:10), function(num_grupo){

  hc_simple <- hcut(distancia_euclidea, k=num_grupo, hc_method = "single")$silinfo$avg.width
  hc_completo <- hcut(distancia_euclidea, k=num_grupo, hc_method = "complete")$silinfo$avg.width
  hc_promedio <- hcut(distancia_euclidea, k=num_grupo, hc_method = "average")$silinfo$avg.width %>%
    mean()

  # crea tabla
  tibble(num_grupo=num_grupo, hc_simple=hc_simple, hc_completo=hc_completo, hc_promedio=hc_promedio)
})

resultados
```

```
## # A tibble: 8 x 4
##   num_grupo hc_simple hc_completo hc_promedio
##   <int>     <dbl>     <dbl>     <dbl>
## 1         3     0.375     0.368     0.368
## 2         4     0.318     0.321     0.321
## 3         5     0.260     0.216     0.270
## 4         6     0.242     0.204     0.222
## 5         7     0.231     0.243     0.235
## 6         8     0.183     0.222     0.207
## 7         9     0.176     0.219     0.193
## 8        10     0.161     0.207     0.181
```

Figura 23. Evaluación del rendimiento de los criterios de enlace (Párraga-Álava, 2020).

2.3 Clustering particional

En el clustering particional el objetivo es obtener una partición de los objetos en grupos o clusters de tal forma que todos los objetos pertenezcan a alguno de los k clusters posibles y que por otra parte los clusters sean disjuntos (Larrañaga et al., 2022).

Algoritmo K-MEANS

El método **K-means** clustering agrupa las observaciones en K clusters, donde el número K lo determina el analista antes de ejecutar del algoritmo. K-means clustering encuentra los K mejores clusters, entendiendo como mejor cluster aquel cuya varianza interna (intra-cluster variation) sea lo más pequeña posible. Se trata por lo tanto de un problema de optimización, en el que se reparten las observaciones en K clusters de forma que la suma de las varianzas internas de todos ellos sea lo menor posible. Para poder solucionar este problema es necesario definir un modo de cuantificar la varianza interna (Garavito, 2022).

Toda observación pertenece al menos a uno de los K clusters. Los clusters no solapan, ninguna observación pertenece a más de un cluster a la vez. En otras palabras, es una partición de los datos (Garavito, 2022). Otras características del algoritmo K-MEDIAS son explicadas por Berzal (2022):

- Algoritmo de agrupamiento por particiones.
- Número de clusters conocido (k).
- Cada clúster tiene asociado un centroide (centro geométrico del cluster).
- Los puntos se asignan al cluster cuyo centroide esté más cerca (utilizando cualquier métrica de distancia).
- Iterativamente, se van actualizando los centroides en función de las asignaciones de puntos a clusters, hasta que los centroides dejen de cambiar.

Respecto al proceso de inicialización y sus iteraciones, (Berzal, 2022) sugiere:

Inicialización.

- Escoger k centroides aleatoriamente (hay métodos más sofisticados).
- Formar k grupos, asignando cada punto al centroide más cercano

Proceso iterativo. Mientras que los centroides cambien:

- Calcular las distancias de todos los puntos a los k centroides.
- Formar k grupos, asignando cada punto al centroide más cercano.
- Recalcular los nuevos centroides

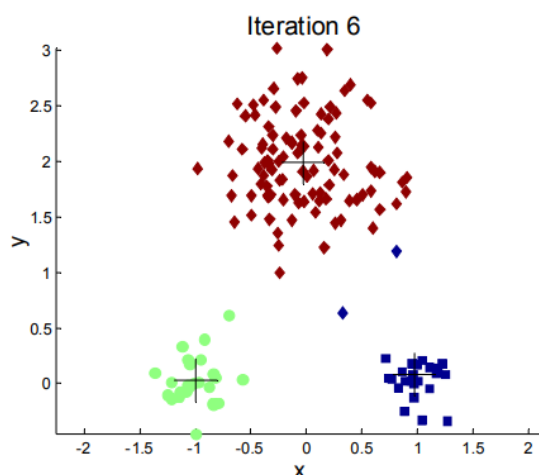


Figura 24. Última iteración del algoritmo K-MEDIAS (Berzal, 2022):

EJEMPLO DE ALGORITMO PARTICIONAL – K-MEANS

Para este ejemplo, se usarán los paquetes, tidyverse, magrittr, dplyr, amap, cluster, factoextra, purrr y NbClust, tal como se presentan en la figura 25.

```

if(!require(tidyverse)) {install.packages("tidyverse")}
if(!require(magrittr)) {install.packages("magrittr")}
if(!require(dplyr)) {install.packages("dplyr")}
if(!require(ama)) {install.packages("ama")}
if(!require(cluster)) {install.packages("cluster")}
if(!require(factoextra)) {install.packages("factoextra")}
if(!require(purrr)) {install.packages("purrr")}
if(!require(NbClust)) {install.packages("NbClust")}
    
```

Figura 25. Instalar y cargar paquetes (Párraga-Álava, 2020).

En este ejemplo se usará nuevamente el conjunto de datos proteína contiene datos del consumo proteínico de 24 países de Europa almacenados como ingestas de proteínas (en porcentaje) de nueve fuentes principales de alimentos.

Cargamos los datos:

<https://raw.githubusercontent.com/jorgeklz/curso-utm/main/proteina.csv>

```

proteina <- read_csv("https://raw.githubusercontent.com/jorgeklz/curso-utm/main/proteina.csv")
proteina

## # A tibble: 24 x 10
##   pais carne_roja carne_blanca huevos leche pescado cereales almidon nueces
##   <chr>      <dbl>      <dbl>  <dbl> <dbl>  <dbl>    <dbl>  <dbl>  <dbl>
## 1 Alba~      10.1        1.4    0.5   8.9    0.2    42.3    0.6    5.5
## 2 Aust~       8.9        14     4.3  19.9    2.1     28     3.6    1.3
## 3 Bélg~      13.5        9.3    4.1  17.5    4.5    26.6    5.7    2.1
## 4 Bulg~       7.8         6     1.6   8.3    1.2    56.7    1.1    3.7
## 5 Repú~       9.7       11.4    2.8  12.5    2     34.3    5     1.1
## 6 Dina~      10.6       10.8    3.7  25     9.9    21.9    4.8    0.7
## 7 Alem~       8.4       11.6    3.7  11.1    5.4    24.6    6.5    0.8
## 8 Finl~       9.5        4.9    2.7  33.7    5.8    26.3    5.1    1
## 9 Fran~      18         9.9    3.3  19.5    5.7    28.1    4.8    2.4
## 10 Grec~     10.2         3     2.8  17.6    5.9    41.7    2.2    7.8
## # ... with 14 more rows, and 1 more variable: frutas_vegetales <dbl>
    
```

Figura 26. Carga de los datos (Párraga-Álava, 2020).

Los datos en las columnas deben estar estandarizados o escalados. Este proceso hace que cada variable tenga una media cero y una desviación estándar uno. Esto se hace para que una unidad en cada variable represente el mismo grado de diferencia (Párraga-Álava, 2020). Usamos la función **scale** de R-base, pero únicamente a variables numéricas.

```
proteina_escala <- proteina %>%
  select_if(is.numeric) %>%
  mutate_if(is.numeric, scale)
proteina_escala

# Asigna pais como etiqueta de fila
proteina_escala <- proteina_escala %>% select_if(is.numeric)
row.names(proteina_escala) <- proteina$pais
```

```
## # A tibble: 24 x 9
##   carne_roja[,1] carne_blanca[,1] huevos[,1] leche[,1] pescado[,1] cereales[,1]
##   <dbl>          <dbl>          <dbl>    <dbl>    <dbl>    <dbl>
## 1      0.0992      -1.73      -2.14    -1.12    -1.19      0.876
## 2     -0.253       1.73       1.27     0.394   -0.640    -0.445
## 3       1.10       0.438      1.09     0.0632  0.0516   -0.574
## 4     -0.577     -0.468     -1.16    -1.21   -0.899     2.21
## 5    -0.0184       1.01    -0.0785  -0.626  -0.669     0.137
## 6       0.246      0.850      0.729   1.10     1.61    -1.01
## 7     -0.400      1.07      0.729  -0.820   0.311   -0.759
## 8    -0.0771    -0.770    -0.168   2.30     0.426   -0.602
## 9       2.42      0.603      0.370   0.339   0.397   -0.436
## 10      0.129     -1.29    -0.0785  0.0770  0.455     0.820
## # ... with 14 more rows, and 3 more variables: almidon[,1] <dbl>,
## #   nueces[,1] <dbl>, frutas_vegetales[,1] <dbl>
```

Figura 27. Estandarización de los datos (Párraga-Álava, 2020).

Luego de contar con los datos, se debe identificar la distancia o similitud entre objetos, por ello procedemos a calcular la matriz de distancia entre cada una de las 24 instancias (países) de la base de datos. Usamos la función **Dist()** del paquete **amap**.

```
# Euclidean
distancia_euclidean <- proteina_escala %>%
  Dist(method = "euclidean", diag = FALSE, upper = TRUE) %>% as.matrix()

# Manhattan
distancia_manhattan <- proteina_escala %>%
  Dist(method = "manhattan", diag = FALSE, upper = TRUE) %>% as.matrix()

# Correlación de Pearson
distancia_pearson <- proteina_escala %>%
  Dist(method = "correlation", diag = FALSE, upper = TRUE) %>% as.matrix()
```

Figura 28. Medidas de distancias/similitud (Párraga-Álava, 2020).

Visualizamos las distancias en forma gráfica con la función **fviz_dist()** del paquete **factoextra**.

```
fviz_dist(as.dist(distancia_euclidea), show_labels = TRUE,
          gradient = list(low = "#00AFBB", mid = "white", high = "#FC4E07"))
```

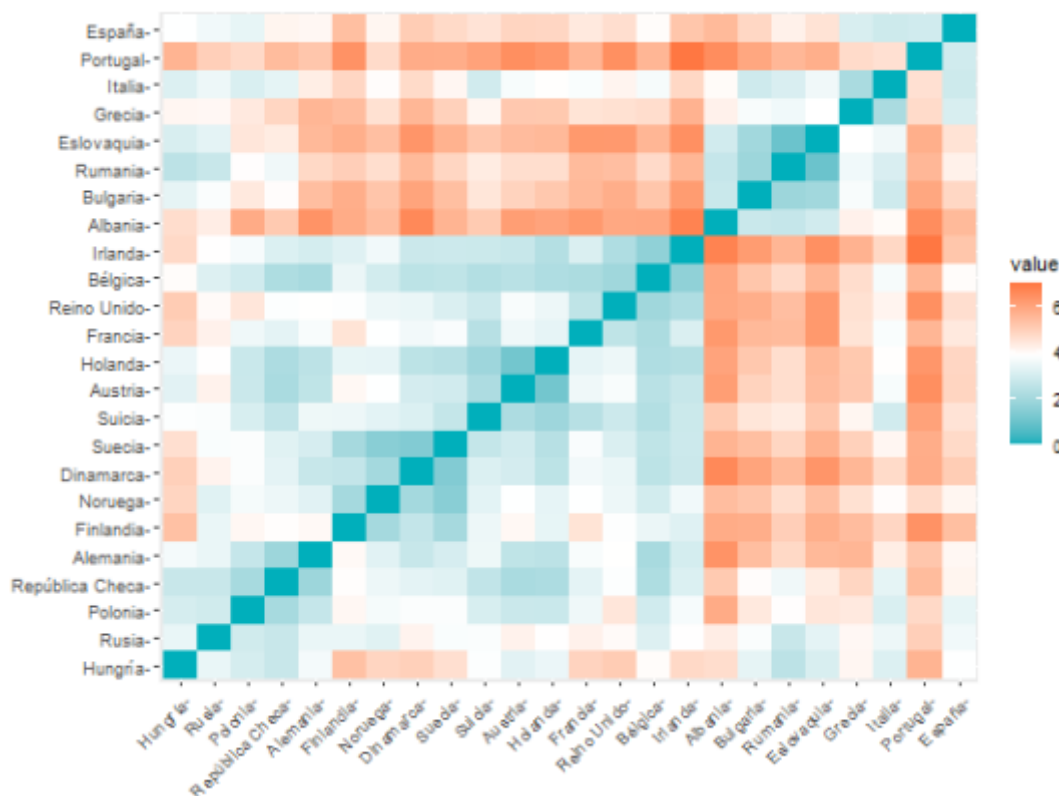


Figura 29. Graficar las medidas de distancias/similitud (Párraga-Álava, 2020).

Ahora aplicamos el algoritmo k-means con la función **amap::Kmeans()** del paquete **amap**.

```
# Se fija la semilla para garantizar reproducibilidad sobre todo en la
# generación de los centroides aleatorios.
set.seed(1987)

# Ejecutamos el algoritmo para el caso de 3 grupos.
km_algoritmo <- Kmeans(proteina_escalas, centers=3,
                      iter.max = 100, method = "euclidean")

km_algoritmo
```

Figura 30.1. Ejecutar algoritmo K-MEANS (Párraga-Álava, 2020).


```
## K-means clustering with 3 clusters of sizes 4, 10, 10
##
## Cluster means:
##   carne_roja carne_blanca   huevos   leche   pescado   cereales
## 1  1.74495815  0.2801323  1.1777053  0.5253350 -0.0420194 -0.6526260
## 2 -0.64728763 -0.7173674 -0.8232721 -0.8485827 -0.1846452  0.8748882
## 3 -0.05069563  0.6053145  0.3521900  0.6384487  0.2014530 -0.6138378
##   almidon   nueces frutas_vegetales
## 1  0.6711480 -0.3811335  0.01357727
## 2 -0.4992134  1.0159395  0.31499256
## 3  0.2307542 -0.8634861 -0.32042347
##
## Clustering vector:
##   Albania   Austria   Bélgica   Bulgaria República Checa
##   2         3         1         2         3
##   Dinamarca   Alemania   Finlandia   Francia   Grecia
##   3         3         3         1         2
##   Hungría   Irlanda   Italia   Holanda   Noruega
##   2         1         2         3         3
##   Polonia   Portugal   Rumania   España   Suecia
##   3         2         2         2         3
##   Suicia   Reino Unido   Rusia   Eslovaquia
##   3         1         2         2
##
## Within cluster sum of squares by cluster:
## [1] 1.802473 4.566422 3.776229
```

Figura 30.2. Ejecutar algoritmo K-MEANS (Párraga-Álava, 2020).

A partir de los resultados, construimos la matriz de partición:

```
# identifica la asignación en grupos de cada objeto
orden_grupos <- order(km_algoritmo$cluster)

# reordena los objetos de acuerdo con su orden en grupos
objeto=proteina$pais[orden_grupos]

# identifica el grupo de cada objeto
grupo=km_algoritmo$cluster[orden_grupos]

# crea matriz de partición
agrupamiento <- as_tibble(cbind(objeto,grupo))

agrupamiento
```

```
## # A tibble: 24 x 2
##   objeto      grupo
##   <chr>      <chr>
## 1 Bélgica    1
## 2 Francia    1
## 3 Irlanda    1
## 4 Reino Unido 1
## 5 Albania    2
## 6 Bulgaria    2
## 7 Grecia      2
## 8 Hungría     2
## 9 Italia      2
## 10 Portugal    2
## # ... with 14 more rows
```

Figura 31. Matriz de partición (Párraga-Álava, 2020).

Determinación de número de grupos

Determinamos el número óptimo de grupos usando el método del codo con métrica de calidad la Silueta, a través de la función `silhouette()` del paquete **cluster**. Para esta evaluación usamos la función `map_dbl()` del paquete **purrr**.

```
set.seed(1987)
# Calcula la silueta para valores k entre 2 y 10
Silueta <- map_dbl(c(2:10), function(i){

  # aplica k-means
  km_algoritmo <- proteina_escala %>%
    Kmeans(centers=i, iter.max = 100, method = "euclidean")
  # computa Silueta
  ss <- silhouette(km_algoritmo$cluster, dist(distancia_euclidea))
  mean(ss[, 3])

})
# Plot el promedio del índice silueta
plot(2:10, Silueta, type = "b", pch = 19, frame = FALSE,
     xlab = "Número de grupos", ylim=c(0,1))
# Dibuja línea vertical
abline(v = 5, lty = 2)
```

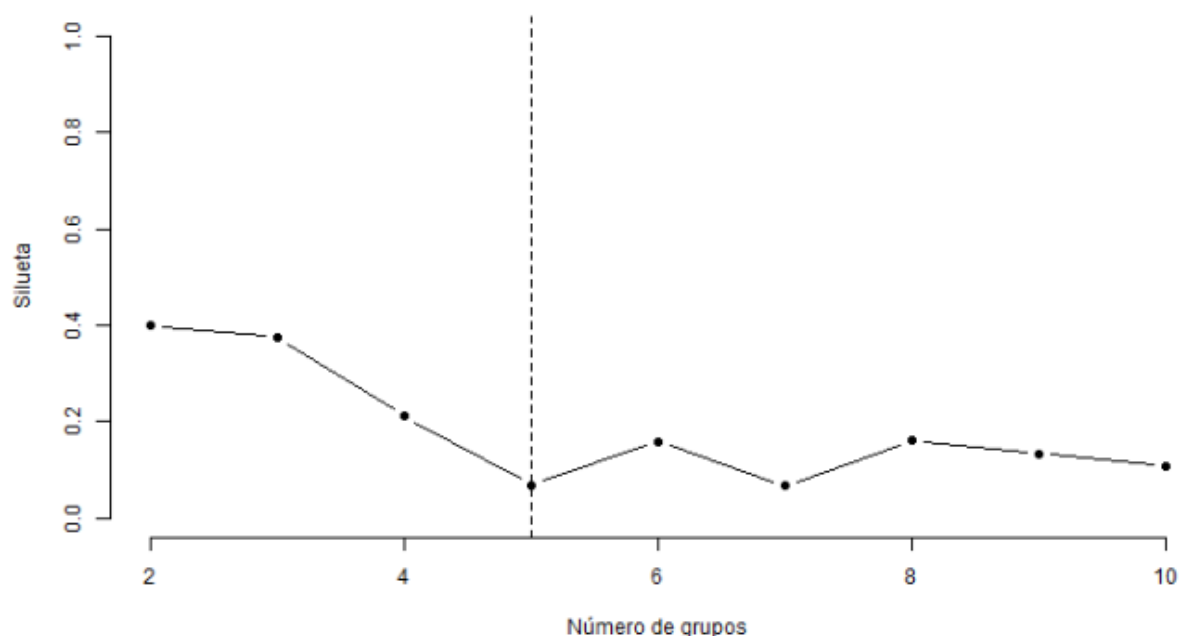


Figura 31. Cálculo y gráfico de la silueta (Párraga-Álava, 2020).

Además de la Silueta, existen varias métricas de calidad que permiten determinar el número óptimo de grupos. La función **NbClust()** del paquete **NbClust** facilita el cálculo simultáneo de 30 métricas para determinar el valor óptimo de k (número de grupos).

```
set.seed(1987)
evalua_k <- proteina_escala %>%
  NbClust(diss = distancia_euclidea, distance = NULL,
          min.nc = 2, max.nc = 10, method = "kmeans", index = "all")
```

Figura 31. Métricas de calidad (Párraga-Álava, 2020).

Para facilitar la interpretación de resultados, usamos la función **fviz_nbclust()** del paquete **factoextra**.

```
fviz_nbclust(evalua_k) + theme_minimal()
```

```
## Among all indices:
## =====
## * 2 proposed 0 as the best number of clusters
## * 1 proposed 1 as the best number of clusters
## * 6 proposed 2 as the best number of clusters
## * 7 proposed 3 as the best number of clusters
## * 1 proposed 4 as the best number of clusters
## * 1 proposed 5 as the best number of clusters
## * 2 proposed 8 as the best number of clusters
## * 3 proposed 9 as the best number of clusters
## * 3 proposed 10 as the best number of clusters
##
## Conclusion
## =====
## * According to the majority rule, the best number of clusters is 3 .
```

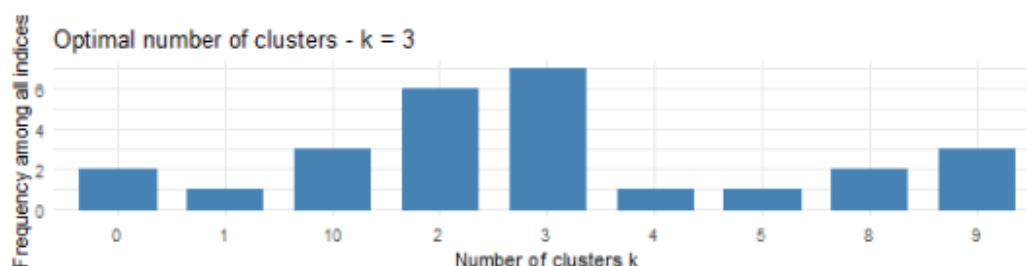


Figura 32. Interpretación de resultados (Párraga-Álava, 2020).

Visualización de grupos

Finalmente, para visualizar los resultados usamos la función **fviz_cluster()** también del paquete **factoextra**.

```

# generamos los grupos visualmente
plot_resul <- fviz_cluster(data = proteina_escalada,
  object= km_algoritmo,
  ellipse.type = "norm",
  palette = c("#2E9FDF", "#00AFBB", "#E7B800"),
  labelsize = 2, main = "Agrupamiento k-means", xlab = "", ylab = "",
)
# agregamos los nombres de países
plot_resul + geom_text(data=plot_resul$data,
  aes(x=x, y=y, label=name, colour=cluster),
  vjust=-1, show.legend = F)
    
```

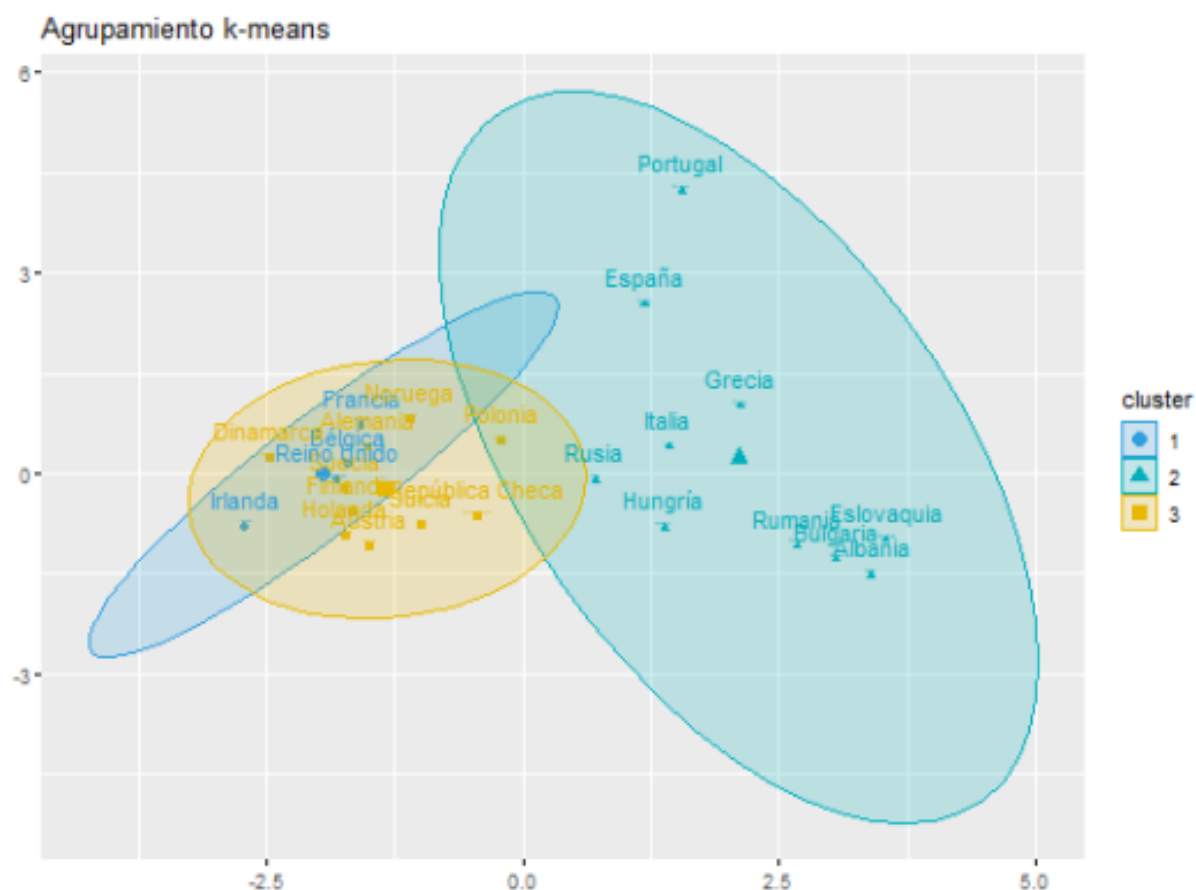


Figura 33. Visualización de grupos (Párraga-Álava, 2020).

Tema 3: Métricas de calidad en modelos descriptivos.

3.1 Homogeneidad.

La noción de homogeneidad define a la igualdad los elementos de un conjunto. Entre más elementos iguales haya en un conjunto, más homogéneo será este conjunto. En las estadísticas, se utiliza la homogeneidad para describir las propiedades estadísticas de un conjunto de datos en particular. En esencia, esto afirma que las propiedades estadísticas de cualquier parte de un conjunto de datos son los mismos que cualquier otra parte. La distancia euclídea será, en consecuencia, recomendable cuando las variables sean homogéneas y estén medidas en unidades similares y/o cuando se desconozca la matriz de varianzas.

3.2 Separación.

Como el objetivo del clustering es agrupar objetos similares en el mismo clúster y objetos diferentes ubicarlos en diferentes clústeres, las métricas de validación interna están basadas usualmente en los criterios de cohesión y separación. En el criterio de cohesión el miembro de cada clúster debe ser lo más cercano posible a los otros miembros del mismo clúster. Mientras que, en el criterio de separación, los clústeres deben estar ampliamente separados entre ellos. Existen varios enfoques para medir esta distancia entre clúster: distancia entre el miembro más cercano, distancia entre los miembros más distantes o la distancia entre los centroides.

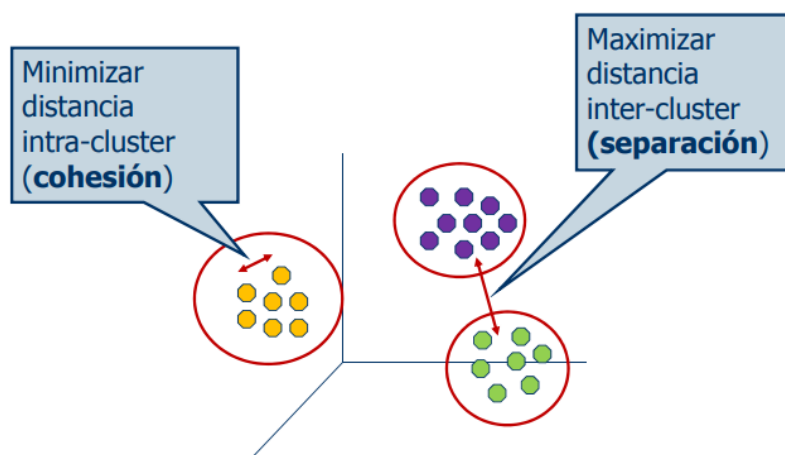


Figura 34. Diferencia entre cohesión y separación (Berzal, 2022).

Coeficiente de Silhouette.

Dado un punto x del conjunto de datos:

- Cohesión $a(x)$: distancia promedio de x a todos los demás puntos en el mismo clúster.
- Separación $b(x)$: distancia promedio de x a todos los demás puntos en el clúster más cercano.

3.3 Jaccard.

El índice Jaccard o coeficiente de correlación Jaccard es similar al simple matching coefficient (SMC). La diferencia radica en que el SMC tiene el término $M_{00}M_{00}$ en el numerador y denominador, mientras que el índice de Jaccard no. Esto significa que, SMC, considera como coincidencias tanto si el atributo está presente en ambos sets como si el atributo no está en ninguno de los sets, mientras que Jaccard solo cuenta como coincidencias cuando el atributo está presente en ambos sets (Joaquín, 2018b).

$$J = \frac{M_{11}}{M_{01} + M_{10} + M_{11}}$$

o en términos matemáticos de sets:

$$J(A, B) = \frac{A \cap B}{A \cup B}$$

La distancia de Jaccard ($1 - J$) supera a la *simple matching distance* en aquellas situaciones en las que la coincidencia de ausencia no aporta información. Para ilustrar este hecho, supóngase que se quiere cuantificar la similitud entre dos clientes de un supermercado en base a los artículos comprados. Es de esperar que cada cliente solo adquiera unos pocos artículos de los muchos disponibles, por lo que el número de artículos no comprados por ninguno ($M_{00}M_{00}$) será muy alto.

Como la distancia de Jaccard ignora las coincidencias de tipo $M_{00}M_{00}$, el grado de similitud dependerá únicamente de las coincidencias entre los artículos comprados (Joaquín, 2018b).

Bibliografía:

- Arroyo-Hernández, J. (2016). Métodos de reducción de dimensionalidad: Análisis comparativo de los métodos APC, ACPP y ACPK. *Uniciencia*, 30(1), 115–122. <https://doi.org/10.15359/ru.30-1.7>
- Beltrán Martínez, B. (2022). *Minería de datos*. <http://bbeltran.cs.buap.mx/NotasMD.pdf>
- Berzal, F. (2022). *Clustering basado en particiones*. [https://elvex.ugr.es/idbis/dm/slides/41 Clustering - Partitional.pdf](https://elvex.ugr.es/idbis/dm/slides/41%20Clustering%20-%20Partitional.pdf)
- Calvo, D. (2018). *Clúster Jerárquicos y No Jerárquicos*. <https://www.diegocalvo.es/cluster-jerarquicos-y-no-jerarquicos/>
- DaSCI. (2022). *Minería de datos descriptiva*. <https://dasci.es/es/linea-investigacion/mineria-de-datos-descriptiva/>
- Garavito, D. (2022). *Agrupamiento (clustering)*. http://rstudio-pubs-static.s3.amazonaws.com/474357_50a1f0fed64d4428b99216cd460a4896.html#02_k-means
- Jiménez Cuadrillero, M. Á. (2018). *Clustering Jerárquico en R*. <https://rpubs.com/mjimcua/clustering-jerarquico-en-r>
- Joaquín Amat, R. (2018a). *Reglas de asociación y algoritmo Apriori con R*. https://www.cienciadedatos.net/documentos/43_reglas_de_asociacion
- Joaquín Amat, R. (2018b). *Sistemas de recomendación con R*. https://www.cienciadedatos.net/documentos/39_sistemas_de_recomendacion_con_r
- Klenzi, R. O., Malberti, M. A., Beguerí, G. E., Klenzi, R. O., Malberti, M. A., & Beguerí, G. E. (2018). Visualización en un entorno de minería de datos desde una perspectiva interacción humano computador. *Computación y Sistemas*, 22(1), 279–290. <https://doi.org/10.13053/CYS-22-1-2558>
- Lafuente, A. (2018). *Reducción de la dimensionalidad (o por qué más datos no*

siempre es mejor) - Aukera. <https://aukera.es/blog/reduccion-dimensionalidad/>

Larrañaga, P., Inza, I., & Moujahid, A. (2022). *Clustering*.

<http://www.sc.ehu.es/ccwbayes/docencia/mmcc/docs/t14clustering.pdf>

Logicalis. (2014). *El data mining y la importancia de la visualización de datos*.

<https://blog.es.logicalis.com/analytics/el-data-mining-y-la-importancia-de-la-visualizacion-de-datos>

Marin Castro, H. M. (2022). *Minería de Datos*.

<https://www.tamps.cinvestav.mx/~hmarin/Mineria/EC2.pdf>

Molina López, J. M., & García Herrero, J. (2006). *Técnicas de análisis de datos*.

http://matema.ujaen.es/jnavas/web_recursos/archivos/weka_master_recursos_naturales/apuntesAD.pdf

Párraga-Álava, J. (2020). *Curso de Introducción a la Minería de Datos con R*.

Sáenz López, A., Cortés Martínez, F., & Betancourt Chávez, J. R. (2017). *Reglas de asociación en una base de datos del área médica*.

<https://www.redalyc.org/journal/1939/193954081005/html/>

Schab, E., Rivera, R., Bracco, L., Coto, F., Cristaldo, P., Ramos, L., Rapesta, N., Pablo Núñez, J., Retamar, S., Casanova, C., De Battista, A., & Herrera, N. E. (2018). *Minería de Datos y Visualización de Información*.

<https://core.ac.uk/download/pdf/296401308.pdf>

Softtek. (2021). *La Reducción de Dimensionalidad en el Machine Learning*.

<https://softtek.eu/tech-magazine/artificial-intelligence/la-reduccion-de-dimensionalidad-en-el-machine-learning/>

Solis, D. (2016). *RPubs - Ejemplo de Reducción de Dimensionalidad con la Base de Datos Iris*. <https://rpubs.com/dsolis/iris-pca-lda>

Soriano Maganto, Á. (2016). *Estudio de técnicas supervisadas de reducción de dimensionalidad para problemas de clasificación*.

<https://core.ac.uk/download/pdf/288499061.pdf>